

**PENGEMBANGAN SISTEM PEMBANGKITAN
DAN PENYIMPANAN AUDIT TRAIL PADA
SISTEM RME DECMED**

Laporan Tugas Akhir

Disusun sebagai syarat kelulusan tingkat sarjana

Oleh

Pradipta Rafa Mahesa

NIM: 13522162



**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO & INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG**

Agustus 2026

**PENGEMBANGAN SISTEM PEMBANGKITAN
DAN PENYIMPANAN AUDIT TRAIL PADA
SISTEM RME DECMED**

Laporan Tugas Akhir

Oleh

Pradipta Rafa Mahesa

NIM: 13522162

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

Telah disetujui dan disahkan sebagai Laporan Tugas Akhir
di Bandung, pada tanggal 25 Agustus 2026

Pembimbing,

Dr. Phil. Eng. Hari Purnama, S.Si., M.Si

NIP. 118110072

LEMBAR PERNYATAAN

Dengan ini saya menyatakan bahwa:

1. Pengerjaan dan penulisan Laporan Tugas Akhir ini dilakukan tanpa menggunakan bantuan yang tidak dibenarkan.
2. Segala bentuk kutipan dan acuan terhadap tulisan orang lain yang digunakan di dalam penyusunan laporan tugas akhir ini telah dituliskan dengan baik dan benar.
3. Laporan Tugas Akhir ini belum pernah diajukan pada program pendidikan di perguruan tinggi mana pun.

Jika terbukti melanggar hal-hal di atas, saya bersedia dikenakan sanksi sesuai dengan Peraturan Akademik dan Kemahasiswaan Institut Teknologi Bandung bagian Penegakan Norma Akademik dan Kemahasiswaan khususnya Pasal 2.1 dan Pasal 2.2.

Bandung, 25 Agustus 2026

Pradipta Rafa Mahesa

NIM 13522162

ABSTRAK

Lorem ipsum dolor sit amet . Operator grafis dan tipografi mengetahui hal ini dengan baik, pada kenyataannya semua profesi yang berhubungan dengan alam semesta komunikasi memiliki hubungan yang stabil dengan kata-kata ini, tetapi apa itu? Lorem ipsum adalah teks dummy tanpa arti. Ini adalah urutan kata Latin yang, sebagaimana posisinya, tidak membentuk kalimat dengan pengertian yang utuh, tetapi memberikan kehidupan pada teks uji yang berguna untuk mengisi ruang-ruang yang selanjutnya akan ditempati dari teks ad hoc yang disusun oleh para profesional komunikasi.

Ini tentu saja yang paling terkenal teks placeholder bahkan jika ada versi berbeda yang dapat dibedakan dari urutan pengulangan kata-kata Latin. Lorem ipsum berisi tipografi yang lebih banyak digunakan, sebuah aspek yang memungkinkan Anda untuk memiliki gambaran umum tentang rendering teks dalam hal pilihan font dan d ukuran font. Jika mengacu pada Lorem ipsum, ekspresi yang digunakan berbeda, yaitu fill text , fiktif text , blind text atau placeholder text : singkatnya, artinya juga bisa nol, tetapi kegunaannya sangat jelas untuk bertahan selama berabad-abad dan menolak versi ironis dan modern yang datang dengan kedatangan web.

Kata kunci: Lorem, Ipsum, Lorem Ipsum

KATA PENGANTAR

Puji dan syukur penulis panjatkan kepada Tuhan Yang Maha Esa atas berkat dan rahmatnya, laporan tugas akhir yang berjudul "PENGEMBANGAN SISTEM PEMBANGKITAN DAN PENYIMPANAN AUDIT TRAIL PADA SISTEM RME DECMED " dapat diselesaikan dalam rangka memenuhi syarat kelulusan tingkat sarjana. Perlu diakui pengerjaan tugas akhir ini didukung oleh banyak pihak. Khususnya, penulis ingin mengucapkan terima kasih kepada:

1. Bapak Dr. Phil. Eng. Hari Purnama, S.Si., M.Si, selaku dosen pembimbing atas segala bentuk dukungan yang telah diberikan dan kesabarannya dalam membimbing penulis serta memberikan saran dalam pengerjaan tugas akhir.
2. Bapak Yudistira Dwi Wardhana Asnar, S.T, Ph.D dan Dr. Agung Dewandaru, S.T., M.Sc., selaku dosen penguji atas segala masukan dan kritik yang telah diberikan terhadap tugas akhir penulis.
3. Dicky Prima Satya, S.T, M.T., Bapak Adi Mulyanto, S.T, M.T., Robithoh Annur, S.T., M.Eng., Ph.D., dan Tricya Esterina Widagdo, ST., M.Sc. selaku dosen koordinator tim tugas akhir atas usahanya mengingatkan mahasiswa program studi Teknik Informatika untuk mengerjakan tugas akhirnya.
4. Seluruh dosen program studi Teknik Informatika ITB yang telah memberikan ilmu pengetahuan yang sangat berharga bagi penulis.
5. Keluarga penulis, terutama orang tua dan kakak penulis, yang senantiasa memberikan doa, dukungan moral, serta dukungan lainnya selama penulis menjalani masa perkuliahan hingga menyelesaikan Tugas Akhir ini.
6. Teman-teman dan rekan seperjuangan mahasiswa bimbingan Bapak Dr. Phil. Eng. Hari Purnama, S.Si., M.Si terutamanya Grup DecMed loh dek, yaitu Bagas dan Tazki, sebagai teman perjuangan selama pengembangan DecMed.

7. Sahabat-sahabat terdekat dari grup "Keluarga Acu", dan "mrt gym" yang telah memberikan bantuan, semangat, serta menjadi tempat berdiskusi dalam menghadapi berbagai proses akademik maupun nonakademik selama menempuh pendidikan di Teknik Informatika ITB.
8. Seluruh pihak lain yang tidak bisa disebutkan disini yang telah membantu dalam proses pengerjaan tugas akhir.

Akhir kata, penulis mengucapkan terima kasih kepada semua pihak yang telah terlibat dalam pengerjaan tugas akhir ini. Penulis juga ingin menyampaikan mohon maaf apabila terdapat kesalahan maupun kekurangan dalam laporan tugas akhir ini. Penulis berharap semoga tugas akhir ini dapat bermanfaat bagi pembaca dan riset-riset kedepannya.

Bandung, 25 Agustus 2026

Pradipta Rafa Mahesa

DAFTAR ISI

ABSTRAK	iv
KATA PENGANTAR.....	v
DAFTAR ISI.....	vii
DAFTAR LAMPIRAN.....	x
DAFTAR GAMBAR.....	xi
DAFTAR TABEL	xii
BAB I PENDAHULUAN	1
I.1 Latar Belakang	1
I.2 Rumusan Masalah	3
I.3 Tujuan dan Ukuran Keberhasilan Pencapaian	3
I.4 Batasan Masalah	4
I.5 Metodologi	4
I.6 Sistematika Pembahasan.....	5
BAB II STUDI LITERATUR	6
II.1 Rekam Medis Elektronik	6
II.1.1 Definisi dan Regulasi RME di Indonesia	6
II.1.2 Kebutuhan Keamanan dan Akuntabilitas Data RME.....	7
II.2 Audit Trail.....	8
II.2.1 Definisi dan Konsep Audit Trail.....	8
II.2.2 Karakteristik Audit Trail yang Baik.....	9
II.2.3 Komponen Audit Trail	10
II.2.4 Kebutuhan Sistem Audit Trail	15
II.2.5 Mekanisme Integritas dan Non-Repudiation pada Audit Trail	17
II.3 Threat Modeling dengan STRIDE	17
II.3.1 Konsep dan Kategori STRIDE	17
II.3.2 Dekomposisi Sistem dan Data Flow Diagram dalam Threat Modeling	18
II.3.3 Pendekatan STRIDE-per-Interaction.....	19
II.3.4 Penurunan Kebutuhan Audit Event dari Analisis Ancaman ..	21
II.4 Kontrol Akses	22

II.4.1	Definisi dan Jenis Kontrol Akses	22
II.4.2	Keterkaitan Kontrol Akses dan Audit Trail dalam Tata Ke- lola Akses.....	22
II.5	Distributed Ledger Technology dan Immutability	23
II.5.1	DLT sebagai Basis Pencatatan Tidak Berubah	23
II.5.2	IOTA	24
II.6	Arsitektur Layanan Pendukung dalam Sistem Terdesentralisasi	25
II.6.1	Pola Layanan Terpisah dalam Arsitektur DecMed	25
II.6.2	Proxy Re-Encryption dan IPFS.....	26
II.7	Penelitian Terkait	27
BAB III	ANALISIS PERSOALAN DAN RANCANGAN SOLUSI	29
III.1	Analisis	29
III.1.1	Analisis Masalah	29
III.1.2	Dekomposisi Aplikasi	31
III.1.3	Identifikasi Ancaman DecMed	41
III.1.4	Analisis Kebutuhan Audit Event	44
III.1.5	Analisis Kebutuhan Sistem	49
III.2	Rancangan	60
III.2.1	Arsitektur Sistem Audit Trail	60
III.2.2	Mekanisme Pembangkitan dan Penerimaan Audit Event	61
III.2.3	Mekanisme Pembentukan dan Penyimpanan Audit Record ..	62
III.2.4	Mekanisme Rotasi, Pengarsipan, dan Publikasi Metadata	63
III.2.5	Pemenuhan Kebutuhan Sistem	64
BAB IV	IMPLEMENTASI DAN PENGUJIAN.....	67
IV.1	Lingkungan Implementasi	67
IV.2	Implementasi	68
IV.2.1	Implementasi Skema Tipe Data Audit	68
IV.2.2	Implementasi Audit Receiver	69
IV.2.3	Implementasi Audit Logger	70
IV.2.4	Implementasi Log Rotation dan Archival Worker	71
IV.2.5	Implementasi Integrasi ATS ke DecMed.....	73
IV.2.6	Implementasi Verifikasi Integritas Audit Record	75
IV.3	Pengujian	76
IV.4	Pengujian	76

BAB V	PENUTUP.....	77
V.1	Kesimpulan.....	77
V.2	Saran.....	77
DAFTAR PUSTAKA		79

DAFTAR LAMPIRAN

Lampiran A. Contoh gambar pengujian

80

DAFTAR GAMBAR

Gambar III.1. Data Flow Diagram Sistem DecMed.....	41
Gambar A.1. Contoh gambar	80

DAFTAR TABEL

Tabel II.1. Kategori Ancaman STRIDE dan Properti Keamanan yang Di- langgar	18
Tabel II.2. Relevansi Kategori STRIDE per Tipe Interaksi.....	20
Tabel III.1. Dependensi Eksternal Sistem DecMed	33
Tabel III.2. Titik Masuk Sistem DecMed	34
Tabel III.3. Aset Sistem DecMed	38
Tabel III.4. Level Kepercayaan Sistem DecMed	39
Tabel III.5. Identifikasi Ancaman Sistem DecMed.....	42
Tabel III.6. Pemetaan Ancaman ke Audit Event.....	44
Tabel III.7. Skema Umum Atribut Audit.....	47
Tabel III.8. Atribut Audit Tambahan untuk Setiap Audit Event.....	47
Tabel III.9. Identifikasi Komponen Infrastruktur Audit Trail berdasarkan ISO 27789 Annex B	50
Tabel III.10. Analisis Gap Infrastruktur Audit Trail terhadap DecMed	52
Tabel III.11. Analisis Kebutuhan Infrastruktur Audit Trail berdasarkan NIST SP 800-53 Rev. 5.....	55
Tabel III.12. Kebutuhan Fungsional Sistem Audit Trail	59
Tabel III.13. Kebutuhan Non-Fungsional Sistem Audit Trail	59
Tabel III.14. Atribut Umum Audit Event	62
Tabel III.15. Pemetaan Kebutuhan ke Mekanisme Rancangan ATS.....	64
Tabel IV.1. Dependensi Utama Audit Trail Service	67
Tabel IV.2. Daftar Implementasi Sistem Audit Trail	68
Tabel IV.3. Pemetaan Event Source terhadap Audit Event yang Diproduksi ..	75

BAB I

PENDAHULUAN

I.1 Latar Belakang

Rekam Medis Elektronik (RME) merupakan komponen fundamental dalam penyelenggaraan pelayanan kesehatan di Indonesia. Peraturan Menteri Kesehatan Republik Indonesia (Permenkes) Nomor 24 Tahun 2022 tentang Rekam Medis mewajibkan seluruh fasilitas pelayanan kesehatan (fasyankes) untuk menyelenggarakan rekam medis dalam bentuk elektronik, serta mengharuskan penyelenggaraan tersebut menjamin kerahasiaan, keutuhan (integritas), dan ketersediaan data rekam medis (Kementerian Kesehatan Republik Indonesia, 2022). Tinjauan sistematis terhadap implementasi RME turut menunjukkan bahwa keamanan dan kerahasiaan data medis pasien menjadi perhatian utama dalam berbagai studi, mengingat data tersebut bersifat sensitif dan berdampak langsung pada kepercayaan pasien terhadap sistem pelayanan kesehatan (Pramesti dkk., 2024). Kewajiban ini menegaskan bahwa keamanan data RME bukan sekadar kebutuhan teknis, melainkan juga kepatuhan regulasi yang mengikat bagi seluruh penyelenggara pelayanan kesehatan.

Untuk menjawab tantangan tersebut, penelitian sebelumnya mengembangkan DecMed, sebuah kerangka kerja manajemen RME terdesentralisasi yang menggunakan Distributed Ledger Technology (DLT) IOTA sebagai basis kontrol akses (Purnama dkk., 2026). DecMed mengintegrasikan Capability-Based Access Control (CapBAC), Proxy Re-Encryption (PRE), dan InterPlanetary File System (IPFS) (Benet, 2014) untuk memastikan bahwa setiap akses terhadap data RME hanya dapat dilakukan atas persetujuan aktif pasien, dengan durasi dan cakupan akses yang dapat dibatasi oleh pasien sendiri (Purnama dkk., 2026).

Meskipun DecMed berhasil menjawab persoalan kontrol akses dan kerahasiaan data, penelitian tersebut secara eksplisit menyatakan bahwa pengelolaan RME juga membutuhkan *audit trail* yang andal dan dapat diverifikasi (*reliable and verifiable audit trails*), karena pencatatan log merupakan komponen inti dari kontrol audit dan umumnya disyaratkan untuk mendukung kepatuhan regulasi maupun investigasi in-

siden (Purnama dkk., 2026). Klaim ini sejalan dengan kerangka kontrol audit pada NIST Special Publication 800-53 Revisi 5, yang menempatkan pencatatan dan peninjauan peristiwa keamanan (kontrol keluarga *Audit and Accountability*) sebagai salah satu persyaratan dasar tata kelola keamanan sistem informasi (National Institute of Standards and Technology, 2020), sejalan pula dengan kontrol *logging* pada ISO/IEC 27002:2022 (International Organization for Standardization, 2022). Catatan audit yang lengkap juga berperan penting sebagai sumber data untuk mendeteksi dan menyelidiki potensi penyalahgunaan oleh pihak internal (*insider threat*) (Theis dkk., 2022), serta menjadi prasyarat akuntabilitas pada mekanisme akses darurat (*break-glass access*) yang lazim dibutuhkan dalam layanan kesehatan (Al Amin dkk., 2025).

Namun demikian, mekanisme pencatatan yang tersedia pada DecMed saat ini belum sepenuhnya memenuhi karakteristik audit trail yang baik. Terdapat dua celah utama yang teridentifikasi. *Pertama*, sebagian besar mekanisme yang tercatat pada jaringan IOTA merupakan entri *capability* atau izin akses (*access grant*), yaitu metadata yang menyatakan siapa berhak mengakses data apa dan sampai kapan, bukan catatan atas peristiwa akses yang benar-benar terjadi (*access event*).. Akibatnya, sistem belum dapat merekonstruksi riwayat siapa yang benar-benar membaca atau mengubah data RME pada waktu tertentu. *Kedua*, sejumlah operasi pada DecMed, seperti pencabutan akses (*revoke_access*) dan pembaruan data administratif maupun klinis (*update_administrative_data*, *update_medical_record*), menyebabkan entri lama pada state tergantikan oleh entri baru alih-alih ditambahkan sebagai riwayat baru yang bersifat *append-only*. Kondisi ini bertentangan dengan karakteristik dasar audit trail, yaitu integritas dan *non-repudiation*, yang mensyaratkan bahwa setiap catatan peristiwa harus bersifat permanen dan tidak dapat diubah maupun dihapus setelah dicatat (National Institute of Standards and Technology, 2020).

Celah tersebut menjadi lebih krusial apabila ditinjau dari perspektif ancaman keamanan sistem. Metode STRIDE (*Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege*) merupakan salah satu metode pemodelan ancaman yang umum digunakan untuk mengidentifikasi risiko keamanan pada suatu sistem informasi secara sistematis, yang kemudian dapat dilanjutkan dengan penilaian risiko menggunakan model DREAD (Laksono dan Prayudi, 2021). Salah satu kategori ancaman pada STRIDE, yaitu *repudiation*, merupakan

kondisi ketika pengguna sistem dapat menyangkal bahwa ia telah melakukan suatu tindakan tertentu; ancaman ini secara langsung berkaitan dengan ketiadaan atau ketidaklengkapan pencatatan log dan mekanisme audit pada suatu sistem (Laksono dan Prayudi, 2021). Dengan kata lain, kebutuhan pencatatan peristiwa (*event*) pada suatu sistem idealnya tidak ditentukan secara ad hoc, melainkan diturunkan dari analisis ancaman yang relevan terhadap arsitektur sistem yang bersangkutan.

Berdasarkan pemaparan tersebut, Tugas Akhir ini bermaksud menganalisis kebutuhan audit trail pada sistem DecMed melalui pendekatan pemodelan ancaman STRIDE, guna menentukan jenis peristiwa (*event*) apa saja yang perlu dicatat agar sistem memiliki audit trail yang memenuhi karakteristik integritas dan non-repudiation. Hasil analisis tersebut selanjutnya digunakan sebagai dasar perancangan dan implementasi sebuah layanan audit trail (*audit trail service*) yang terintegrasi dengan arsitektur DecMed yang telah ada, mengikuti pola layanan pendukung (*supporting service*) yang serupa dengan *proxy re-encryption server* yang sudah digunakan pada DecMed (Purnama dkk., 2026). Metadata dari setiap peristiwa yang tercatat akan disimpan pada jaringan IOTA untuk menjamin integritas dan keterlacakannya, sedangkan berkas log peristiwa akan disimpan pada IPFS (Benet, 2014), konsisten dengan pola pemisahan penyimpanan on-chain dan off-chain yang telah diterapkan pada DecMed untuk data RME (Purnama dkk., 2026).

I.2 Rumusan Masalah

Berdasarkan latar belakang yang telah dipaparkan pada Subbab I.1, terdapat dua rumusan masalah utama dalam Tugas Akhir ini, yaitu

1. Peristiwa (*event*) apa saja yang perlu dicatat oleh mekanisme audit trail pada sistem DecMed?
2. Bagaimana rancangan dan implementasi layanan audit trail yang sesuai dengan arsitektur DecMed?

I.3 Tujuan dan Ukuran Keberhasilan Pencapaian

Tujuan dari Tugas Akhir ini adalah

1. Menghasilkan daftar kebutuhan peristiwa (*event requirements*) audit trail pada sistem DecMed berdasarkan analisis ancaman menggunakan metode STRIDE.
2. Merancang dan mengimplementasikan layanan audit trail yang terintegrasi dengan arsitektur DecMed.

Ukuran keberhasilan pencapaian Tugas Akhir ini didasarkan pada kesesuaian daftar kebutuhan *event* yang dihasilkan terhadap kategori ancaman STRIDE yang teridentifikasi, serta keberhasilan layanan audit trail yang dikembangkan dalam mencatat, menyimpan, dan menyajikan kembali peristiwa-peristiwa tersebut secara benar dan tidak dapat diubah (*immutable*).

I.4 Batasan Masalah

Terdapat beberapa batasan masalah dari Tugas Akhir ini, antara lain

1. Analisis ancaman menggunakan STRIDE dilakukan terhadap arsitektur sistem DecMed yang sudah ada, bukan terhadap perancangan ulang sistem DecMed secara keseluruhan;
2. Layanan audit trail yang dikembangkan bersifat pencatat pasif (*passive recorder*) yang menerima dan menyimpan peristiwa, tanpa melakukan validasi otorisasi maupun keputusan kontrol akses terhadap peristiwa yang dicatat; dan
3. Implementasi dilakukan pada lingkungan pengembangan yang sama dengan yang digunakan pada penelitian DecMed sebelumnya.

I.5 Metodologi

Metodologi yang digunakan dalam Tugas Akhir ini terdiri dari empat tahap pelaksanaan, sebagai berikut.

1. **Tahap Perencanaan.** Tahap ini berfokus pada pemahaman arsitektur dan mekanisme sistem DecMed yang telah ada, kajian literatur terkait audit trail, threat modeling STRIDE, serta penelitian terkait lainnya sebagai dasar analisis.

2. **Tahap Analisis Ancaman.** Pada tahap ini dilakukan identifikasi aset, aliran data, dan batas kepercayaan (*trust boundary*) pada arsitektur DecMed, dilanjutkan dengan penerapan metode STRIDE untuk mengidentifikasi ancaman pada setiap komponen. Hasil identifikasi ancaman tersebut kemudian diturunkan menjadi daftar kebutuhan peristiwa (*event requirements*) yang perlu dicatat oleh audit trail.
3. **Tahap Perancangan dan Implementasi.** Berdasarkan daftar kebutuhan *event* yang telah ditentukan, dilakukan perancangan arsitektur layanan audit trail beserta skema penyimpanan metadata pada IOTA dan berkas log pada IPFS. Selanjutnya, rancangan tersebut diimplementasikan sebagai layanan yang terintegrasi dengan komponen-komponen DecMed yang sudah ada.
4. **Tahap Pengujian dan Evaluasi.** Tahap terakhir bertujuan menguji dan mengevaluasi apakah layanan audit trail yang dikembangkan berhasil mencatat seluruh *event* yang telah didefinisikan, serta memvalidasi bahwa data yang tercatat memenuhi karakteristik integritas dan non-repudiation yang diharapkan.

I.6 Sistematika Pembahasan

Berikut merupakan sistematika pembahasan dari Tugas Akhir ini.

Bab I menjelaskan gagasan utama dari Tugas Akhir ini yang meliputi latar belakang, rumusan masalah, tujuan, batasan, metodologi, hingga sistematika pembahasan.

Bab II memaparkan studi literatur yang dilakukan, mencakup RME, audit trail, threat modeling STRIDE, kontrol akses, DLT dan IOTA, serta teknologi pendukung lain yang digunakan pada DecMed.

Bab III menjelaskan analisis ancaman menggunakan STRIDE terhadap sistem DecMed serta rancangan solusi layanan audit trail yang diusulkan.

Bab IV menjelaskan hasil implementasi dari rancangan yang dipaparkan pada Bab III, beserta mekanisme dan hasil pengujian yang dilakukan.

Bab V merupakan bab penutup yang berisi kesimpulan dan saran dari penulis.

BAB II

STUDI LITERATUR

II.1 Rekam Medis Elektronik

II.1.1 Definisi dan Regulasi RME di Indonesia

Sesuai dengan Pasal 1 ayat (1) dan (2) Peraturan Menteri Kesehatan Republik Indonesia Nomor 24 Tahun 2022 tentang Rekam Medis, rekam medis adalah dokumen yang berisikan data identitas pasien, pemeriksaan, pengobatan, tindakan, dan pelayanan lain yang telah diberikan kepada pasien, sedangkan Rekam Medis Elektronik (RME) adalah rekam medis yang dibuat dengan menggunakan sistem elektronik yang diperuntukkan bagi penyelenggaraan rekam medis (Kementerian Kesehatan Republik Indonesia, 2022). Berdasarkan definisi tersebut, RME bukan sekadar bentuk digital dari berkas rekam medis konvensional, melainkan mencakup keseluruhan proses penyelenggaraan yang berkelanjutan, mulai dari bagaimana data dibuat, diakses, disimpan, hingga didistribusikan kepada pihak-pihak tertentu untuk memenuhi kebutuhan pelayanan kesehatan. Permenkes Nomor 24 Tahun 2022 juga mewajibkan seluruh fasilitas pelayanan kesehatan (fasyankes) di Indonesia, mulai dari praktik mandiri tenaga kesehatan hingga rumah sakit, untuk menyelenggarakan RME sebagai pengganti rekam medis konvensional (Kementerian Kesehatan Republik Indonesia, 2022).

Selain mewajibkan bentuk elektronik, Permenkes Nomor 24 Tahun 2022 turut mengatur hak pasien atas data rekam medis miliknya, termasuk larangan akses oleh pihak yang tidak berwenang (Kementerian Kesehatan Republik Indonesia, 2022). Ketentuan ini menjadi dasar regulasi bahwa setiap akses terhadap data RME seorang pasien pada dasarnya harus melalui mekanisme yang dapat mempertanggungjawabkan siapa yang mengakses, kapan, dan untuk keperluan apa. Hal tersebut sejalan dengan Undang-Undang Nomor 19 Tahun 2016 tentang Perubahan atas Undang-Undang Nomor 11 Tahun 2008 tentang Informasi dan Transaksi Elektronik, yang mengategorikan RME sebagai dokumen elektronik sehingga harus dibuat dan dikelola

dalam sistem elektronik dengan jaminan keamanan dan akuntabilitas agar dapat digunakan sebagai alat bukti yang sah secara hukum (Pemerintah Republik Indonesia, 2016). Dengan kata lain, regulasi di Indonesia tidak hanya menuntut RME diselenggarakan secara elektronik, tetapi juga menuntut adanya mekanisme yang menjamin kerahasiaan dan keutuhan data tersebut sepanjang siklus penyelenggaraannya, sebuah kebutuhan yang menjadi dasar bagi berbagai penelitian pengembangan sistem manajemen RME yang berfokus pada keamanan data, salah satunya DecMed (Purnama dkk., 2026).

II.1.2 Kebutuhan Keamanan dan Akuntabilitas Data RME

Data RME tergolong sebagai data yang sensitif karena memuat informasi identitas dan kondisi kesehatan pasien, sehingga keamanan dan kerahasiaannya menjadi perhatian utama dalam berbagai penelitian sistem informasi kesehatan. Tinjauan sistematis terhadap implementasi RME menunjukkan bahwa aspek keamanan dan kerahasiaan data medis pasien secara konsisten menjadi isu sentral, dan kegagalan menjaga aspek tersebut berdampak langsung pada menurunnya kepercayaan pasien terhadap sistem pelayanan kesehatan yang digunakan (Pramesti dkk., 2024). Kerahasiaan data RME tidak dapat dijaga hanya dengan membatasi siapa saja yang boleh mengakses data (kontrol akses), tetapi juga membutuhkan mekanisme yang dapat menunjukkan bahwa pembatasan tersebut benar-benar dipatuhi dan dapat ditelusuri kembali ketika terjadi insiden.

Kebutuhan penelusuran kembali (*traceability*) inilah yang menjadikan akuntabilitas sebagai aspek keamanan yang tidak dapat dipisahkan dari kerahasiaan dan integritas data RME. Akuntabilitas dalam konteks ini merujuk pada kemampuan suatu sistem untuk menghubungkan setiap tindakan yang dilakukan terhadap data dengan identitas pihak yang melakukannya, sehingga setiap pihak yang mengakses maupun mengubah data RME dapat dimintai pertanggungjawaban atas tindakannya. Sebagai contoh, kerangka kerja DecMed yang menjadi basis pengembangan pada Tugas Akhir ini dirancang agar seluruh akses terhadap data RME hanya dapat dilakukan atas persetujuan aktif pasien melalui mekanisme kontrol akses berbasis *capability* (Purnama dkk., 2026). Namun demikian, tanpa mekanisme yang menjamin akuntabilitas, kontrol akses yang ketat sekalipun tidak dapat membuktikan bahwa suatu pelanggaran keamanan benar-benar terjadi, siapa pihak yang bertanggung jawab,

maupun bagaimana kronologi pelanggaran tersebut (Purnama dkk., 2026). Kebutuhan inilah yang menjadi latar belakang pentingnya mekanisme audit trail sebagai pelengkap dari kontrol akses pada sistem manajemen RME, yang akan dibahas lebih lanjut pada Subbab II.2.

II.2 Audit Trail

II.2.1 Definisi dan Konsep Audit Trail

Audit trail didefinisikan sebagai rekaman kronologis atas aktivitas sistem yang cukup untuk memungkinkan rekonstruksi, peninjauan, dan pemeriksaan urutan kejadian yang mengelilingi atau menyebabkan suatu operasi, prosedur, atau peristiwa dalam suatu transaksi, sejak awal hingga hasil akhirnya (International Organization for Standardization, 2021). Definisi ini menegaskan bahwa audit trail bukan sekadar kumpulan catatan yang berdiri sendiri, melainkan suatu rangkaian catatan yang secara kolektif membentuk kronologi yang dapat ditelusuri ulang, sehingga setiap catatan di dalamnya perlu memiliki keterkaitan yang jelas dengan catatan lain dari kejadian yang sama. Standar yang sama juga membedakan secara eksplisit antara *audit record* sebagai catatan satu peristiwa tunggal dalam siklus hidup suatu rekam medis elektronik, *audit log* sebagai kumpulan audit record yang tersusun secara kronologis, dan *audit trail* sebagai kumpulan audit log yang secara keseluruhan membentuk riwayat lengkap suatu entitas (International Organization for Standardization, 2021).

Konsep audit trail perlu dibedakan secara tegas dari konsep kontrol akses maupun pencatatan izin akses (*access grant log*). Audit trail bersifat komplementer terhadap kontrol akses, bukan pengganti maupun bagian dari kontrol akses itu sendiri: kontrol akses menentukan kebijakan mengenai siapa *berhak* mengakses data, sedangkan audit trail mencatat apa yang *benar-benar terjadi* pada sistem, termasuk apabila terjadi penyimpangan dari kebijakan tersebut (International Organization for Standardization, 2021). Perbedaan ini penting untuk digarisbawahi karena suatu sistem dapat saja memiliki kontrol akses yang ketat dan mencatat metadata pemberian izin secara lengkap, namun tetap tidak memiliki audit trail yang memadai apabila tidak ada catatan terpisah mengenai peristiwa akses aktual (*access event*) yang terjadi setelah izin tersebut diberikan. Dengan kata lain, audit trail idealnya mencatat peristiwa-

wa (*event*) sebagai representasi dari tindakan nyata yang terjadi pada sistem, bukan semata representasi dari kebijakan atau otorisasi yang berlaku.

II.2.2 Karakteristik Audit Trail yang Baik

Agar dapat berfungsi sebagai bukti yang dapat diandalkan, audit trail perlu memenuhi sejumlah karakteristik tertentu. Karakteristik pertama adalah kemampuan mengidentifikasi pengguna secara tidak ambigu (*unambiguous identification*), yaitu audit trail harus menyediakan data yang cukup untuk mengidentifikasi secara pasti setiap pengguna maupun sistem eksternal yang telah mengakses atau menerima data rekam medis (International Organization for Standardization, 2021). Karakteristik kedua adalah integritas (*integrity*) dan kerahasiaan (*confidentiality*) dari audit trail itu sendiri, yaitu jaminan bahwa catatan yang telah dibuat terlindungi dari upaya perusakan (*tampering*), termasuk perlunya audit trail mencatat setiap tindakan yang dilakukan terhadap audit trail itu sendiri, seperti kapan sistem audit tidak berfungsi atau dinonaktifkan (International Organization for Standardization, 2021).

Karakteristik lain yang tidak kalah penting adalah kelengkapan (*completeness*) dan ketersediaan (*availability*). Kelengkapan berarti audit trail harus mencatat seluruh tindakan yang relevan terhadap data – termasuk pembuatan, pembacaan, pembauran, maupun pengarsipan – setiap kali tindakan tersebut dilakukan terhadap data kesehatan pribadi (International Organization for Standardization, 2021), sedangkan ketersediaan berarti sistem audit harus memastikan entri tercatat setiap kali sistem informasi kesehatan sedang beroperasi (International Organization for Standardization, 2021). Selain keempat karakteristik tersebut, audit trail yang baik juga perlu mendukung kemampuan deteksi terhadap potensi penyalahgunaan oleh pihak internal (*insider threat*), mengingat pelaku insider pada umumnya memiliki akses sah terhadap sistem sehingga satu-satunya cara untuk mendeteksi maupun membuktikan penyalahgunaan tersebut adalah melalui pencatatan aktivitas yang menyeluruh dan tidak dapat dimanipulasi oleh pelaku itu sendiri (Theis dkk., 2022). Karakteristik-karakteristik tersebut menjadi acuan utama dalam menilai apakah suatu mekanisme pencatatan pada sistem informasi, termasuk pada DecMed, telah dapat disebut sebagai audit trail yang memadai.

II.2.3 Komponen Audit Trail

Untuk dapat berfungsi secara menyeluruh, suatu sistem audit trail perlu dibangun melalui beberapa komponen fungsional yang saling berurutan. Kerangka kerja audit trail untuk rekam medis elektronik membagi persoalan menjadi penentuan peristiwa apa saja yang perlu dipicu untuk dicatat (*trigger events*), format dan isi dari setiap catatan yang dihasilkan, hingga bagaimana catatan tersebut dikelola secara aman sepanjang siklus hidupnya (International Organization for Standardization, 2021). Pembagian yang serupa juga ditemukan pada kerangka manajemen log komputer secara umum, yang memandang manajemen log sebagai suatu infrastruktur yang terdiri atas perangkat keras, perangkat lunak, jaringan, dan media yang digunakan untuk menghasilkan (*generate*), mengirimkan (*transmit*), menyimpan (*store*), serta menganalisis (*analyze*) data log (Kent dan Souppaya, 2006). Berdasarkan kedua kerangka tersebut, komponen audit trail pada Tugas Akhir ini dikelompokkan menjadi empat bagian, yaitu penentuan event, pengumpulan event, penyimpanan event, dan pembacaan event, sebagaimana diuraikan pada subbagian berikut.

II.2.3.1 Penentuan Event

Komponen pertama adalah penentuan peristiwa (*event*) apa saja yang perlu memicu pencatatan audit trail. Peristiwa pemicu (*trigger events*) pada sistem rekam medis elektronik pada umumnya ditentukan berdasarkan skala, tujuan, serta kebijakan privasi dan keamanan dari masing-masing sistem informasi kesehatan, dengan cakupan minimal berupa peristiwa akses (*access events*) dan peristiwa kueri (*query events*) terhadap data kesehatan pribadi (International Organization for Standardization, 2021). Standar tersebut turut memberikan contoh peristiwa yang secara eksplisit berada di luar cakupan audit trail rekam medis, seperti peristiwa mulai/berhenti aplikasi, peristiwa autentikasi pengguna, maupun peristiwa yang dihasilkan oleh sistem operasi (International Organization for Standardization, 2021).

Di luar peristiwa akses dan kueri, penentuan trigger events pada suatu sistem juga dapat diturunkan melalui pendekatan pemodelan ancaman (*threat modeling*), seperti metode STRIDE, yang mengidentifikasi ancaman keamanan pada suatu sistem secara sistematis dan dapat digunakan sebagai dasar penentuan kebutuhan pencatatan (Laksono dan Prayudi, 2021).

II.2.3.2 Konten Wajib dan Konten Tambahan pada Event

Setiap audit record, terlepas dari jenis peristiwa apa yang dicatatnya, memuat sekumpulan bidang data minimal yang bersifat wajib (*mandatory*). Terdapat sembilan bidang yang wajib diisi pada setiap event apa pun jenisnya, yaitu identitas unik peristiwa (*EventID*), jenis tindakan yang dilakukan (*EventActionCode*), waktu terjadinya peristiwa (*EventDateTime*), identitas pengguna atau proses pelaku tindakan (*UserID*), identitas sumber audit atau sistem yang menghasilkan catatan tersebut (*AuditSourceID*), serta empat bidang yang secara bersama-sama mengidentifikasi objek data yang menjadi sasaran tindakan (*ParticipantObjectTypeCode*, *ParticipantObjectTypeCodeRole*, *ParticipantObjectTypeCode*, dan *ParticipantObjectID*) (International Organization for Standardization, 2021).

Di luar konten wajib tersebut, terdapat pula sejumlah bidang bersifat opsional (*optional*) maupun kondisional yang dapat ditambahkan sesuai kebutuhan spesifik dari masing-masing jenis event. Sebagai contoh, pada event akses (*access events*) yang mencakup pembuatan, pembacaan, pembaruan, dan penghapusan data, bidang *EventActionCode* dibatasi pada empat nilai tertentu (*Create*, *Read*, *Update*, *Delete*) dan objek yang diaudit secara spesifik merepresentasikan data pasien (International Organization for Standardization, 2021). Sebaliknya, pada event kueri (*query events*), dibutuhkan bidang tambahan yang tidak diperlukan pada event akses, yaitu identitas pihak yang mengajukan kueri (*Questioner related*), pihak yang merespons kueri (*Question ahead related*), serta pihak lain yang turut terlibat (*Alternative participant related*), dan bidang *ParticipantObjectQuery* yang pada event kueri berubah sifatnya dari opsional menjadi kondisional mandatory karena perlu menyimpan isi kueri yang sesungguhnya diajukan (International Organization for Standardization, 2021). Bidang opsional lain yang lazim ditambahkan pada berbagai jenis event antara lain indikator keberhasilan tindakan (*EventOutcomeIndicator*), peran pengguna saat melakukan tindakan (*RoleIDCode*), tujuan penggunaan data (*PurposeOfUse*), maupun tingkat sensitivitas objek yang diakses (*ParticipantObjectSensitivity*) (International Organization for Standardization, 2021).

II.2.3.3 Pengumpulan Event

Komponen kedua adalah pengumpulan event, yaitu bagaimana setiap peristiwa yang terjadi pada sistem direkam menjadi suatu catatan (*audit record*) dengan format dan

isi yang konsisten. Format umum suatu audit record pada rekam medis elektronik sekurang-kurangnya memuat identitas unik dari peristiwa yang terjadi (*EventID*), jenis tindakan yang dilakukan (*EventActionCode*), waktu terjadinya peristiwa (*EventDateTime*), identitas pengguna atau proses yang melakukan tindakan (*UserID*), serta identitas objek data yang menjadi sasaran tindakan tersebut (*ParticipantObjectID*) (International Organization for Standardization, 2021).

Pengumpulan event pada praktiknya menghadapi tantangan tersendiri, terutama pada sistem dengan banyak sumber log (*log sources*) yang berbeda. Setiap sumber log cenderung mencatat informasi dengan format dan kelengkapan yang berbeda-beda, sehingga menyulitkan proses menghubungkan (*correlate*) peristiwa yang tercatat oleh satu sumber dengan peristiwa yang tercatat oleh sumber lainnya (Kent dan Souppaya, 2006).

II.2.3.4 Penyimpanan Event

Komponen ketiga adalah penyimpanan event, yaitu bagaimana catatan peristiwa yang telah dikumpulkan disimpan dan dikelola secara aman sepanjang siklus hidupnya. Sistem audit trail perlu menyediakan langkah pengamanan yang cukup untuk melindungi audit log dari perusakan (*tampering*), termasuk mengamankan akses terhadap audit record, mengamankan akses terhadap perangkat audit itu sendiri, serta mencatat seluruh tindakan yang dilakukan terhadap audit trail dengan menyertakan waktu, jenis tindakan, dan pelakunya (International Organization for Standardization, 2021). Selain itu, audit record juga perlu tunduk pada kebijakan retensi (*retention policy*) yang idealnya disesuaikan dengan masa berlaku data rekam medis yang bersangkutan, mengingat audit record dapat dibutuhkan sebagai bukti dalam jangka waktu yang panjang (International Organization for Standardization, 2021).

Pada level infrastruktur, pertimbangan penyimpanan log turut mencakup keseimbangan antara volume data log yang dihasilkan dengan kapasitas penyimpanan yang tersedia, baik untuk penyimpanan daring (*online*) maupun luring (*offline*), serta kebutuhan keamanan atas data tersebut (Kent dan Souppaya, 2006).

II.2.3.5 Pembacaan Event

Komponen keempat adalah pembacaan event, yaitu bagaimana catatan peristiwa yang telah tersimpan dapat diakses kembali dan dianalisis oleh pihak yang berwenang. Akses terhadap audit data harus dikendalikan secara ketat dan itu sendiri menjadi objek audit (*itself subject to audit*), dengan akses yang idealnya dilakukan melalui sistem informasi yang dapat menegakkan kontrol tersebut, bukan akses langsung terhadap audit trail (International Organization for Standardization, 2021). Fasilitas pembacaan audit trail idealnya juga mendukung analisis berdasarkan kombinasi berbagai bidang data yang tercatat, seperti seluruh akses oleh pengguna tertentu, seluruh tindakan penghapusan yang dilakukan oleh pengguna dengan peran tertentu, atau seluruh peristiwa yang melibatkan data pasien tertentu dalam rentang waktu tertentu (International Organization for Standardization, 2021).

Kemampuan pembacaan yang fleksibel semacam ini menjadi penting bagi pihak yang bertanggung jawab atas suatu sistem untuk dapat melakukan investigasi maupun evaluasi kepatuhan kebijakan secara efektif, karena analisis log pada dasarnya merupakan proses berkelanjutan yang perlu dilakukan secara berkala, bukan hanya ketika terjadi insiden (Kent dan Souppaya, 2006).

II.2.3.6 Hash-Chaining

Hash-chaining adalah mekanisme di mana setiap catatan dalam sebuah urutan menyertakan nilai *hash* kriptografis dari catatan sebelumnya sebagai bagian dari isinya sendiri (Islam dan Rahman, 2025). Dengan demikian, setiap catatan secara kriptografis terikat pada seluruh catatan sebelumnya dalam rantai.

Properti utama yang dihasilkan oleh *hash-chaining* adalah kemampuan deteksi perubahan: apabila isi satu catatan dimodifikasi, nilai *hash* catatan tersebut akan berubah, yang pada gilirannya menyebabkan nilai *hash* yang disimpan di catatan sesudahnya menjadi tidak konsisten (Islam dan Rahman, 2025). Efek domino ini berlanjut ke seluruh catatan sesudahnya dalam rantai, sehingga modifikasi pada posisi mana pun dalam rantai akan terdeteksi melalui pemeriksaan konsistensi nilai *hash*. Mekanisme ini menjadikan *hash-chaining* sebagai pendekatan yang efektif untuk menegakkan sifat *append-only* pada sistem pencatatan.

II.2.3.7 Tanda Tangan Digital untuk Keaslian dan Non-Repudiation

Tanda tangan digital merupakan mekanisme kriptografis yang memungkinkan suatu entitas membuktikan kepemilikan atas sebuah pesan atau dokumen. Prinsip kerjanya adalah: pengirim menandatangani data menggunakan *private key* miliknya, menghasilkan tanda tangan yang dapat diverifikasi oleh siapa pun yang memiliki *public key* bersesuaian (shostack2014).

Dalam konteks audit trail, tanda tangan digital memenuhi dua kebutuhan sekaligus. *Pertama*, keaslian (*authenticity*): karena hanya pemegang *private key* yang dapat menghasilkan tanda tangan yang valid untuk kunci publik tertentu, verifikasi tanda tangan membuktikan bahwa audit event benar-benar dibangkitkan oleh komponen yang mengklaim sebagai sumbernya. *Kedua*, non-repudiation: apabila suatu entitas telah menandatangani sebuah event, entitas tersebut tidak dapat menyangkal pernah membangkitkan event tersebut selama *private key*-nya tidak dikompromikan (shostack2014).

Salah satu skema tanda tangan digital yang banyak digunakan untuk keperluan ini adalah Ed25519, yang berbasis kurva eliptik Edwards25519. Skema ini bersifat deterministik (tanda tangan yang dihasilkan selalu sama untuk input yang sama), memiliki ukuran kunci dan tanda tangan yang kecil (32 byte untuk kunci publik, 64 byte untuk tanda tangan), dan efisien secara komputasi, menjadikannya pilihan yang sesuai untuk lingkungan yang membutuhkan verifikasi tanda tangan dalam volume tinggi.

II.2.3.8 Rotasi dan Pengarsipan Log

Rotasi log adalah praktik manajemen log di mana berkas log aktif secara berkala diganti dengan berkas baru, sementara berkas lama diarsipkan untuk keperluan retensi (Kent dan Souppaya, 2006). Praktik ini memiliki tiga manfaat utama dalam konteks audit trail.

Pertama, manajemen ukuran: berkas log yang terus tumbuh tanpa batas akan menyulitkan pengelolaan dan dapat menghabiskan kapasitas penyimpanan. Dengan rotasi berkala, setiap berkas arsip memiliki ukuran yang dapat diprediksi. *Kedua*, efisiensi analisis: investigasi seringkali berfokus pada rentang waktu tertentu; berkas arsip yang terorganisasi berdasarkan periode waktu memudahkan pencarian

data yang relevan tanpa harus memindai seluruh riwayat log. *Ketiga*, keamanan penyimpanan: berkas arsip yang telah dirotasi dapat segera dipindahkan ke media penyimpanan yang lebih aman dan tahan lama, terpisah dari lingkungan operasional yang lebih rentan terhadap gangguan (Kent dan Souppaya, 2006).

Kombinasi antara *hash-chaining* pada level *record*, tanda tangan digital untuk keaslian *event*, dan rotasi log untuk pengarsipan berkala membentuk tiga lapisan mekanisme yang saling melengkapi dalam menjamin integritas, keaslian, dan ketersediaan jangka panjang dari sebuah sistem audit trail.

II.2.4 Kebutuhan Sistem Audit Trail

Berdasarkan definisi, karakteristik, komponen, dan model layanan yang telah diuraikan pada subbab sebelumnya, dapat diidentifikasi kebutuhan-kebutuhan yang harus dipenuhi oleh suatu sistem audit trail agar dapat berfungsi secara efektif sebagai bukti digital dalam proses audit maupun investigasi insiden keamanan. Kebutuhan tersebut dapat dikelompokkan menjadi kebutuhan fungsional dan kebutuhan keamanan.

II.2.4.1 Kebutuhan Fungsional

Kebutuhan fungsional berkaitan dengan kemampuan yang harus dimiliki sistem audit trail untuk menjalankan fungsinya:

- **Pencatatan event yang komprehensif.** Sistem harus mampu mencatat seluruh aktivitas yang relevan dari berbagai komponen sistem, tidak terbatas pada satu jenis aktivitas saja. Cakupan minimal mencakup peristiwa akses dan peristiwa kueri terhadap data sensitif (International Organization for Standardization, 2021).
- **Format record yang konsisten.** Setiap audit record harus memiliki struktur dan atribut yang seragam, memuat setidaknya informasi mengenai jenis event, waktu kejadian, sumber event, hasil event, serta identitas aktor dan objek yang terlibat (International Organization for Standardization, 2021).
- **Penyimpanan terpusat dan terdedikasi.** Seluruh audit record harus tersimpan pada repositori khusus yang terpisah dari data operasional sistem, sehingga dapat diakses secara independen untuk keperluan audit (International

Organization for Standardization, 2021).

- **Kemampuan pencarian dan analisis.** Sistem harus menyediakan fasilitas untuk mengambil dan menganalisis audit record berdasarkan berbagai kombinasi atribut seperti rentang waktu, jenis event, aktor, maupun objek yang terlibat (International Organization for Standardization, 2021).
- **Retensi jangka panjang.** Audit record harus dipertahankan selama periode yang sesuai dengan kebutuhan organisasi dan regulasi yang berlaku, mempertimbangkan bahwa investigasi dapat dilakukan jauh setelah kejadian (International Organization for Standardization, 2021).

II.2.4.2 Kebutuhan Keamanan

Kebutuhan keamanan berkaitan dengan sifat-sifat yang harus dimiliki audit record agar dapat dipercaya sebagai bukti digital:

- **Integritas.** Audit record yang telah tersimpan harus terlindungi dari modifikasi yang tidak sah. Setiap perubahan pada audit record harus dapat terdeteksi (International Organization for Standardization, 2021).
- **Keaslian (*authenticity*).** Harus tersedia mekanisme untuk memverifikasi bahwa audit record benar-benar dibangkitkan oleh komponen sistem yang diklaim sebagai sumbernya, bukan oleh pihak lain yang berpura-pura menjadi sumber tersebut (International Organization for Standardization, 2021).
- **Non-repudiation.** Audit record harus dapat digunakan untuk membuktikan bahwa suatu aktivitas benar-benar terjadi dan dilakukan oleh entitas tertentu, sehingga entitas tersebut tidak dapat menyangkal keterlibatannya (**shostack2014**, International Organization for Standardization, 2021).
- **Sifat *append-only*.** Mekanisme penyimpanan audit record harus bersifat tambah-saja, artinya entri yang telah tercatat tidak dapat diubah maupun dihapus. Perubahan kondisi akibat aktivitas berikutnya dicatat sebagai entri baru yang terpisah, bukan sebagai pembaruan entri lama (International Organization for Standardization, 2021).
- **Akses terkendali.** Akses terhadap audit record harus dikendalikan secara ketat dan akses itu sendiri harus menjadi objek audit, sehingga tidak ada pihak

yang dapat mengakses maupun memanipulasi audit record tanpa meninggalkan jejak (International Organization for Standardization, 2021).

Kebutuhan-kebutuhan tersebut bersifat saling melengkapi: sistem yang hanya memenuhi kebutuhan fungsional tanpa memenuhi kebutuhan keamanan tidak dapat diandalkan sebagai bukti digital, sedangkan sistem yang memenuhi kebutuhan keamanan namun tidak memenuhi kebutuhan fungsional tidak dapat digunakan untuk investigasi yang komprehensif. Kebutuhan inilah yang akan digunakan sebagai tolok ukur dalam mengevaluasi kondisi audit trail pada sistem DecMed di Bab III.

II.2.5 Mekanisme Integritas dan Non-Repudiation pada Audit Trail

Untuk memenuhi kebutuhan keamanan yang telah diidentifikasi pada Subbab II.2.4, khususnya integritas, keaslian, dan non-repudiation, terdapat beberapa mekanisme kriptografis yang lazim digunakan dalam perancangan sistem audit trail.

II.3 Threat Modeling dengan STRIDE

II.3.1 Konsep dan Kategori STRIDE

Threat modeling (pemodelan ancaman) merupakan proses terstruktur untuk mengidentifikasi, mengenumerasi, dan memprioritaskan ancaman keamanan pada suatu sistem sebelum ancaman tersebut benar-benar dieksploitasi (shostack2014). Proses ini umumnya terdiri atas empat pertanyaan pokok: apa yang sedang dibangun, apa yang dapat salah, apa yang perlu dilakukan terhadap hal tersebut, dan apakah pekerjaan tersebut telah dilakukan dengan cukup baik (shostack2014).

STRIDE merupakan salah satu kerangka kerja *threat modeling* yang dikembangkan oleh Loren Kohnfelder dan Praerit Garg di Microsoft, dengan nama yang dibentuk dari huruf pertama masing-masing kategori ancaman (shostack2014). Kerangka kerja ini dirancang untuk membantu pengembang perangkat lunak mengidentifikasi jenis-jenis serangan yang cenderung dialami suatu sistem. Setiap kategori STRIDE merupakan kebalikan dari properti keamanan yang diinginkan pada suatu sistem, sebagaimana ditunjukkan pada Tabel II.1.

Tabel II.1: Kategori Ancaman STRIDE dan Properti Keamanan yang Dilanggar

Kategori	Properti yang Dilanggar	Definisi
Spoofing	Authentication	Berpura-pura menjadi sesuatu atau seseorang selain diri sendiri.
Tampering	Integrity	Memodifikasi sesuatu pada disk, jaringan, atau memori.
Repudiation	Non-repudiation	Mengklaim tidak melakukan sesuatu atau tidak bertanggung jawab atas suatu tindakan.
Information Disclosure	Confidentiality	Memberikan informasi kepada pihak yang tidak berwenang untuk melihatnya.
Denial of Service	Availability	Menyerap sumber daya yang dibutuhkan untuk menyediakan layanan.
Elevation of Privilege	Authorization	Memungkinkan seseorang melakukan sesuatu yang tidak diizinkan untuk dilakukan.

Kategori *Repudiation* memiliki karakteristik yang berbeda dari kategori lainnya: ancaman ini beroperasi pada lapisan bisnis, bukan semata pada lapisan teknis jaringan atau aplikasi (**shostack2014**). Penting untuk dipahami bahwa repudiasi tidak selalu merupakan tindakan jahat, misalnya seseorang dapat secara jujur menyatakan tidak melakukan sesuatu karena kegagalan teknologi atau proses. Oleh karena itu, pertanyaan kunci bagi perancang sistem adalah: *bukti apa yang dimiliki sistem untuk membantah atau membuktikan suatu klaim?* Apabila sistem tidak memiliki log, tidak menyimpan log, atau tidak dapat menganalisis log, ancaman repudiasi menjadi sangat sulit untuk dibantah (**shostack2014**). Hal ini menegaskan keterkaitan langsung antara kategori *Repudiation* pada STRIDE dengan kebutuhan mekanisme audit trail pada suatu sistem.

II.3.2 Dekomposisi Sistem dan Data Flow Diagram dalam Threat Modeling

Sebelum mengidentifikasi ancaman menggunakan STRIDE, suatu sistem perlu didekomposisi terlebih dahulu untuk memperoleh pemahaman menyeluruh mengenai cara kerja sistem dan bagaimana sistem tersebut berinteraksi dengan entitas eksternal (**shostack2014**). Dekomposisi ini umumnya mencakup identifikasi deskripsi sistem, dependensi eksternal, titik masuk (*entry points*), aset yang perlu dilindungi, dan tingkat kepercayaan (*trust level*) dari setiap entitas yang terlibat.

Data Flow Diagram (DFD) merupakan alat utama yang digunakan dalam tahap dekomposisi ini. DFD menggambarkan bagaimana data mengalir di antara elemen-elemen sistem, sehingga memungkinkan analisis untuk mengidentifikasi di mana ancaman keamanan paling mungkin muncul (**shostack2014**). DFD dalam konteks *threat modeling* terdiri atas empat elemen dasar:

- (*Process*) : elemen yang menerima, mengolah, dan mengirimkan data, biasanya direpresentasikan sebagai lingkaran atau persegi panjang dengan sudut membulat.
- (*Data Store*) : tempat data disimpan secara persisten; direpresentasikan sebagai dua garis paralel.
- (*Data Flow*) : perpindahan data antar elemen; direpresentasikan sebagai panah berlabel.
- (*External Entity*) : aktor atau sistem di luar kendali sistem yang dianalisis; direpresentasikan sebagai persegi panjang.

Elemen tambahan yang krusial dalam *threat modeling* adalah **batas kepercayaan** (*trust boundary*), yang memisahkan bagian-bagian sistem dengan tingkat kepercayaan berbeda dan direpresentasikan sebagai garis putus-putus. Ancaman paling signifikan umumnya muncul pada aliran data yang melintasi batas kepercayaan ini, karena pada titik itulah asumsi mengenai kepercayaan terhadap data maupun pengirimnya berubah (**shostack2014**).

II.3.3 Pendekatan STRIDE-per-Interaction

Terdapat dua pendekatan utama dalam menerapkan STRIDE pada DFD suatu sistem: STRIDE-per-element dan STRIDE-per-interaction (**shostack2014**).

STRIDE-per-element merupakan pendekatan yang lebih sederhana, di mana setiap elemen DFD (proses, penyimpanan data, aliran data, entitas eksternal) diperiksa terhadap subset kategori STRIDE yang paling relevan bagi tipe elemen tersebut. Pendekatan ini mudah dipelajari dan dapat digunakan tanpa memerlukan tabel referensi yang kompleks, namun cenderung menghasilkan ancaman yang serupa berulang kali pada model yang besar (**shostack2014**).

STRIDE-per-interaction merupakan pendekatan yang lebih rinci, di mana ancam-

an diperiksa berdasarkan setiap interaksi antar elemen dalam bentuk tuple (asal, tujuan, jenis interaksi), bukan berdasarkan elemen secara individual (**shostack2014**). Pendekatan ini dikembangkan oleh Larry Osterman dan Douglas MacIver di Microsoft. Keunggulan utamanya adalah bahwa ancaman diidentifikasi dalam konteks interaksi nyata yang terjadi pada sistem, sehingga lebih mudah dipahami dan lebih langsung terhubung ke keputusan perancangan yang perlu diambil (**shostack2014**).

Pada STRIDE-per-interaction, tidak semua kategori STRIDE relevan untuk setiap tipe interaksi. Tabel II.2 menunjukkan kategori STRIDE yang relevan untuk setiap tipe interaksi berdasarkan kerangka yang dikembangkan oleh Shostack (**shostack2014**).

Tabel II.2: Relevansi Kategori STRIDE per Tipe Interaksi

Tipe Interaksi	S	T	R	I	D	E
Proses mengirim output ke proses lain	✓	✓	✓	✓		✓
Proses mengirim output ke entitas eksternal (kode)	✓		✓	✓		
Proses mengirim output ke entitas eksternal (manusia)			✓			
Proses menerima input dari penyimpanan data	✓	✓	✓	✓		
Proses menerima input dari proses lain	✓		✓		✓	✓
Proses menerima input dari entitas eksternal	✓		✓		✓	✓
Proses mengirim output ke penyimpanan data		✓	✓	✓	✓	
Aliran data melintasi batas mesin/jaringan	✓	✓		✓	✓	
Penyimpanan data menerima aliran data dari proses		✓	✓	✓	✓	
Penyimpanan data mengirim aliran data ke proses		✓	✓	✓	✓	
Entitas eksternal mengirim input ke proses	✓		✓	✓		
Entitas eksternal menerima input dari proses	✓		✓			

Sebagai ilustrasi penerapan tabel tersebut: ketika sebuah proses menerima *input* dari entitas eksternal, ancaman yang relevan adalah *Spoofing* (entitas eksternal yang mengirim data mungkin bukan entitas yang sebenarnya), *Repudiation* (entitas eksternal dapat menyangkal pernah mengirimkan data tersebut), dan *Elevation of Privilege* (entitas eksternal dapat mengirim data yang memaksa proses melakukan se-

suatu di luar kewenangan pengirim). Sebaliknya, *Tampering* dan *Denial of Service* kurang relevan pada interaksi ini karena proses yang menerima data umumnya sudah memiliki kendali penuh atas cara data tersebut diproses (shostack2014).

STRIDE-per-interaction memerlukan tabel referensi untuk digunakan secara efektif, namun menghasilkan ancaman yang lebih kontekstual dan lebih mudah dihubungkan ke keputusan mitigasi yang spesifik (shostack2014).

II.3.4 Penurunan Kebutuhan Audit Event dari Analisis Ancaman

Hasil identifikasi ancaman menggunakan STRIDE pada suatu sistem selanjutnya dapat dinilai tingkat risikonya menggunakan metodologi DREAD (*Damage potential, Reproducibility, Exploitability, Affected user, Discoverability*), sehingga dapat disusun peringkat ancaman yang menjadi dasar prioritas penyusunan langkah mitigasi (Laksono dan Prayudi, 2021). Setiap kategori ancaman STRIDE pada dasarnya berkaitan dengan bidang keamanan (*security control*) tertentu yang relevan sebagai acuan mitigasinya, misalnya kategori *Spoofing* berkaitan dengan bidang *authentication*, kategori *Tampering* berkaitan dengan bidang *integrity*, dan kategori *Repudiation* berkaitan dengan bidang *non-repudiation* (Laksono dan Prayudi, 2021).

Penerapan STRIDE pada studi kasus sistem informasi akademik menunjukkan keterkaitan langsung antara ancaman kategori *Repudiation* dengan kebutuhan pencatatan log. Salah satu ancaman kategori *Repudiation* yang teridentifikasi pada studi tersebut adalah pencatatan log yang minim sebagai bukti penanganan klaim penyangkalan (Laksono dan Prayudi, 2021). Langkah mitigasi yang diusulkan untuk ancaman kategori *Repudiation* pada studi tersebut mencakup ketentuan bahwa segala tindakan yang dilakukan pada sistem harus dicatat pada log sebagai bahan pembuktian atas terjadinya suatu tindakan, disertai penerapan tanda tangan digital (*digital signature*) yang tercatat secara otomatis pada log setiap kali terjadi perubahan data (Laksono dan Prayudi, 2021).

II.4 Kontrol Akses

II.4.1 Definisi dan Jenis Kontrol Akses

Kontrol akses merupakan mekanisme yang bertujuan membatasi tindakan atau operasi yang dapat dilakukan oleh pengguna sah dalam suatu sistem komputer, termasuk mengatur tindakan dari program yang berjalan atas nama pengguna tersebut. Terdapat beberapa model kontrol akses yang umum digunakan, di antaranya Role-Based Access Control (RBAC), yang mengatur akses pengguna berdasarkan peran (*role*) yang dimilikinya, dan Attribute-Based Access Control (ABAC), yang mengatur akses dengan menilai atribut yang melekat pada subjek, objek, serta kondisi lingkungan yang relevan (Purnama dkk., 2026).

Model kontrol akses lain yang relevan dengan Tugas Akhir ini adalah Capability-Based Access Control (CapBAC). Konsep utama CapBAC adalah *capability*, yaitu suatu objek yang berisi hak akses dan dapat diberikan kepada suatu entitas sehingga entitas tersebut dapat mengakses data milik entitas lain (Hernández-Ramos dkk., 2013). Setiap entitas pemilik data dapat membuat *capability* dan mendistribusikannya secara langsung kepada penerima yang dituju, maupun menyimpannya pada suatu lokasi yang menjamin integritasnya (Purnama dkk., 2026). CapBAC menawarkan kontrol akses dengan granularitas tinggi, karena setiap *capability* dapat memuat berbagai atribut akses seperti metadata pemberi akses, bagian data yang dapat diakses, jenis akses, dan waktu kedaluwarsa akses, serta cocok digunakan pada lingkungan terdesentralisasi karena tidak membutuhkan otoritas terpusat untuk memvalidasi maupun memberikan akses (Purnama dkk., 2026).

II.4.2 Keterkaitan Kontrol Akses dan Audit Trail dalam Tata Kelola Akses

Kontrol akses dan audit trail merupakan dua mekanisme yang berbeda secara konseptual namun saling melengkapi dalam tata kelola akses terhadap suatu sistem. Otorisasi (*authorization*) didefinisikan sebagai pemberian hak, termasuk pemberian akses berdasarkan hak akses (*access rights*) yang dimiliki (International Organization for Standardization, 2021), sedangkan audit trail – sebagaimana telah dibahas pada Subbab II.2.1 – merupakan rekaman kronologis atas aktivitas yang benar-

benar terjadi pada sistem. Kedua konsep tersebut saling terhubung melalui kebijakan akses (*access policy*), yaitu bahwa audit trail dari suatu rekam medis elektronik disyaratkan untuk dapat mendukung kesesuaian terhadap etika profesi, kebijakan organisasi, dan regulasi yang berlaku, meskipun audit trail semata tidak cukup untuk menilai kelengkapan suatu rekam medis elektronik (International Organization for Standardization, 2021).

Keterkaitan antara kontrol akses dan audit trail juga tercermin pada tingkat akses terhadap audit trail itu sendiri. Akses terhadap audit trail disyaratkan untuk diatur oleh suatu kebijakan akses tersendiri, yang ditetapkan oleh organisasi yang bertanggung jawab atas pengelolaan audit log (International Organization for Standardization, 2021). Standar yang sama turut mendefinisikan bidang *Participant Object Permission PolicySet* pada format audit record, yang berfungsi untuk merujuk kebijakan akses aktual yang berlaku pada suatu peristiwa (International Organization for Standardization, 2021), yang menunjukkan bahwa kebijakan kontrol akses dan catatan audit trail pada praktiknya perlu saling terhubung dan tidak dirancang sebagai dua mekanisme yang terpisah sepenuhnya.

II.5 Distributed Ledger Technology dan Immutability

II.5.1 DLT sebagai Basis Pencatatan Tidak Berubah

Distributed Ledger Technology (DLT) merupakan sistem terdesentralisasi yang memungkinkan data direkam dan disimpan pada sejumlah *node* dalam suatu jaringan, sehingga seluruh salinan *ledger* tetap tersinkronisasi dan konsisten satu sama lain (Purnama dkk., 2026). Karakteristik desentralisasi ini menghilangkan kebutuhan akan otoritas terpusat, sehingga meningkatkan transparansi dalam sistem, dan menjadikan DLT banyak digunakan di berbagai sektor, termasuk keuangan, *supply chain*, kesehatan, dan energi (Purnama dkk., 2026).

Salah satu karakteristik utama DLT yang relevan bagi kebutuhan audit trail adalah *immutability*, yaitu sifat data yang telah tersimpan pada *ledger* tidak dapat diubah maupun dihapus setelah tercatat. Karakteristik ini menjadikan DLT sebagai salah satu teknologi yang cocok untuk diintegrasikan dengan mekanisme kontrol akses

berbasis *capability* (CapBAC), karena DLT dapat berfungsi sebagai media penyimpanan bagi *capability* yang membutuhkan jaminan integritas (Purnama dkk., 2026).

II.5.2 IOTA

IOTA merupakan salah satu jenis DLT yang mengadopsi struktur jaringan berbasis *Directed Acyclic Graph* (DAG). Struktur jaringan DAG memungkinkan IOTA mendukung *throughput* yang tinggi, karena mampu memvalidasi transaksi baru secara lebih cepat dibandingkan *blockchain* konvensional seperti Bitcoin maupun Ethereum (Purnama dkk., 2026). Pada versi awalnya (IOTA 2.0), IOTA tidak membebankan biaya transaksi (*zero-cost transactions*), sehingga cocok digunakan untuk transaksi mikro maupun pada lingkungan ringan seperti perangkat Internet of Things (IoT). Namun, pada IOTA Rebased, model transaksi tanpa biaya tersebut tidak lagi berlaku, dan setiap transaksi dikenakan biaya (*gas fee*) (Purnama dkk., 2026).

IOTA Rebased menandai evolusi signifikan dari protokol IOTA, bertransisi dari struktur Tangle yang mendukung transaksi tanpa biaya dan bergantung pada *co-ordinator* terpusat, menuju *object-based ledger* yang dibangun di atas DAG. Berbeda dari IOTA 2.0 yang berbasis model *Unspent Transaction Output* (UTXO), IOTA Rebased mengadopsi DAG berbasis objek yang terinspirasi dari arsitektur Sui Network, sehingga memungkinkan pemrosesan transaksi secara paralel dan mendukung pola transaksi yang kompleks melalui kontrol akses *fine-grained* (Purnama dkk., 2026). *Ledger* pada IOTA Rebased membedakan antara *shared objects*, yang dapat diakses dan dimodifikasi oleh banyak entitas berdasarkan aturan yang telah ditentukan, dengan *owned objects*, yang sepenuhnya dikontrol oleh satu entitas (Purnama dkk., 2026). Konsensus pada IOTA Rebased ditenagai oleh protokol Mysticeti, sebuah sistem *Byzantine Fault Tolerant* (BFT) yang mengimplementasikan *Delegated Proof-of-Stake* (DPoS) untuk mencapai *throughput* tinggi dan *latency* rendah, dengan *finality* dalam waktu sekitar 500 milidetik dan mendukung lebih dari 50.000 transaksi per detik (Purnama dkk., 2026).

Untuk menyederhanakan interaksi pengguna dengan jaringan IOTA, tersedia IOTA Gas Station yang memungkinkan transaksi bersponsor (*sponsored transactions*), sehingga pengguna tidak perlu mengelola *token* untuk membayar biaya transaksi (Purnama dkk., 2026). Arsitektur IOTA Gas Station terdiri atas tiga komponen, yaitu Gas Station Server sebagai komponen utama yang menyediakan API untuk

mereservasi *gas object* dan mengeksekusi transaksi, Gas Station Storage sebagai penyimpanan persisten untuk *gas object* dan status transaksi bersponsor, serta Key Store Manager (KSM) yang mengelola kunci untuk menandatangani transaksi, baik melalui layanan eksternal (*External Key Management Service*) maupun penyimpanan kunci dalam memori (Purnama dkk., 2026).

II.6 Arsitektur Layanan Pendukung dalam Sistem Terdesentralisasi

II.6.1 Pola Layanan Terpisah dalam Arsitektur DecMed

Arsitektur DecMed dirancang mengikuti pola tiga lapis (*three-tier architecture*), yang terdiri atas *Client Layer*, *Proxy Layer*, dan *Storage Layer* (Purnama dkk., 2026). *Client Layer* berfungsi sebagai antarmuka utama bagi seluruh aktor untuk berinteraksi dengan sistem, dengan tiga aplikasi klien berbeda yang disesuaikan dengan kebutuhan masing-masing peran. *Proxy Layer* berperan sebagai *middleware* keamanan yang menegakkan kebijakan kontrol akses dan mengelola operasi kriptografi seperti Proxy Re-Encryption (PRE), sedangkan *Storage Layer* terbagi menjadi penyimpanan *on-chain* yang menjamin keutuhan *capability* akses, dan penyimpanan *off-chain* yang menangani data klinis terenkripsi dalam volume besar (Purnama dkk., 2026).

Lapisan *Proxy* dan *Storage* pada DecMed diimplementasikan melalui beberapa layanan (*server*) yang terpisah satu sama lain, masing-masing dengan tanggung jawab yang spesifik. Layanan *PRE Server* bertanggung jawab menangani proses *re-encryption*, di mana setiap kali menerima permintaan data, *server* tersebut terlebih dahulu memverifikasi entri *capability* pada jaringan IOTA sebelum melakukan *re-encryption* terhadap data yang diambil dari IPFS untuk pemohon, dengan Redis digunakan untuk menyimpan data sementara seperti *nonce* guna mencegah *replay attack* (Purnama dkk., 2026). Layanan *Gas Station Server* bertanggung jawab mensponsori transaksi pada jaringan IOTA, sehingga aktor yang tidak memiliki kepentingan finansial dapat berinteraksi dengan *ledger* tanpa perlu memiliki *token* (Purnama dkk., 2026). Kedua layanan tersebut berkomunikasi dengan jaringan IOTA maupun IPFS sesuai kebutuhannya masing-masing, sementara lapisan penyim-

panan sendiri terbagi menjadi IOTA Tangle sebagai *trust anchor* yang immutable untuk kebijakan kontrol akses (CapBAC), dan IPFS Cluster sebagai penyimpanan *content-addressable* yang terdesentralisasi untuk berkas RME terenkripsi (Purnama dkk., 2026).

II.6.2 Proxy Re-Encryption dan IPFS

Proxy Re-Encryption (PRE) memanfaatkan *proxy* yang bersifat *semi-trusted* untuk mentransformasikan suatu *ciphertext* menjadi *ciphertext* baru yang isinya tetap sama, namun hanya dapat didekripsi menggunakan *secret key* milik subjek yang dituju (Purnama dkk., 2026). *Ciphertext* awal dihasilkan dari enkripsi data menggunakan *public key* milik *delegator* (pemilik data), dan dapat disimpan pada suatu lokasi penyimpanan seperti *cloud storage* maupun sistem DLT. Ketika *delegatee* (penerima akses) meminta akses terhadap data terenkripsi tersebut, *delegator* membuat *re-encryption key* menggunakan *secret key* miliknya dan *public key* milik *delegatee*; *re-encryption key* tersebut dikirimkan kepada *proxy*, yang kemudian menggunakannya bersama *ciphertext* awal dan *public key* kedua belah pihak untuk menghasilkan *ciphertext* baru yang hanya dapat didekripsi oleh *delegatee* (Purnama dkk., 2026). Meskipun PRE membatasi kemampuan *proxy* untuk mengetahui isi data asli (*plaintext*), arsitektur ini tetap memusatkan fungsi *re-encryption* pada satu entitas *semi-trusted*, sehingga menciptakan *single point of failure*: apabila *proxy* mengalami gangguan atau menjadi target serangan *denial-of-service*, seluruh proses *re-encryption* akan terhenti dan alur otorisasi maupun distribusi kunci menjadi tidak tersedia bagi seluruh subjek yang bergantung padanya (Purnama dkk., 2026).

InterPlanetary File System (IPFS) adalah suatu sistem berkas terdistribusi *peer-to-peer* yang bertujuan menghubungkan seluruh perangkat komputasi ke dalam satu sistem berkas (Benet, 2014). Berbeda dengan protokol Hypertext Transfer Protocol (HTTP) konvensional yang menggunakan pengalamatan berbasis lokasi (*location-based addressing*), IPFS menggunakan pengalamatan berbasis konten (*content-based addressing*), yaitu data diidentifikasi dan diambil berdasarkan *hash* kriptografisnya yang disebut *Content Identifier* (CID) (Purnama dkk., 2026). Ketika suatu berkas ditambahkan ke jaringan IPFS, berkas tersebut dipecah menjadi bagian-bagian kecil, di-*hash* secara kriptografis, dan diberikan CID yang unik sebagai penanda permanen dan *immutable* dari data tersebut; apabila isi berkas berubah,

hash kriptografisnya turut berubah dan menghasilkan CID yang sepenuhnya baru, sehingga karakteristik *immutability* ini menjadikan IPFS sangat kompatibel dengan DLT dalam menjaga integritas data (Purnama dkk., 2026). Dalam konteks penyimpanan data kesehatan, IPFS memungkinkan penyimpanan *off-chain* untuk berkas medis terenkripsi berukuran besar, sementara hanya CID berukuran kecil yang perlu disimpan pada *ledger* yang *immutable (on-chain)*, sehingga mengatasi keterbatasan ukuran penyimpanan yang umum ditemukan pada jaringan DLT maupun *blockchain* tanpa mengorbankan arsitektur terdesentralisasi yang bebas dari *single point of failure* (Purnama dkk., 2026).

II.7 Penelitian Terkait

DecMed, sebagai kerangka kerja manajemen RME yang menjadi basis pengembangan Tugas Akhir ini, mengklaim menghasilkan mekanisme kontrol akses yang didukung oleh audit trail yang tahan terhadap perusakan (*tamper-evident audit trails*) (Purnama dkk., 2026). Klaim tersebut didasarkan pada mekanisme bahwa setiap akses yang diberikan oleh pasien dicatat sebagai entri *capability* pada penyimpanan *on-chain* di jaringan IOTA, dan sifat *immutability* dari penyimpanan *on-chain* tersebut menjamin bahwa entri tidak dapat diubah oleh pihak lain, dengan setiap permintaan akses divalidasi terhadap keberadaan entri yang bersesuaian melalui *smart contract* (Purnama dkk., 2026). Peristiwa akses, pemberian persetujuan (*consent*), dan pelanggaran kebijakan pada DecMed dinyatakan tercatat secara *tamper-evident*, sehingga menyediakan audit trail yang dapat diandalkan untuk mendukung akuntabilitas lintas batas organisasi (Purnama dkk., 2026).

Salah satu contoh sistem audit trail yang secara khusus dibangun dengan memanfaatkan kombinasi DLT dan IPFS adalah LogStamping (Islam dan Rahman, 2025). Sistem tersebut menggunakan Ethereum sebagai platform *blockchain* dengan konsensus *Proof of Authority*, di mana log dikelompokkan menjadi beberapa *chunk* berdasarkan jumlah baris maupun interval waktu, kemudian dihitung *hash* SHA-256 dari setiap *chunk* tersebut dan disimpan pada *blockchain*, sementara berkas log asli yang telah terenkripsi dan diverifikasi diarsipkan secara terpisah pada IPFS untuk menjamin keberadaannya secara *tamper-proof* dan persisten (Islam dan Rahman, 2025). Pendekatan tersebut menghasilkan audit trail yang ringan (*lightweight*)

dan dapat diverifikasi, karena *blockchain* hanya menyimpan *hash* sebagai bukti integritas alih-alih menyimpan seluruh data log secara langsung, sedangkan mekanisme verifikasi bekerja dengan menghitung ulang *hash* dari suatu kelompok log dan membandingkannya dengan *hash* yang tersimpan pada *blockchain* untuk mendeteksi adanya perubahan (Islam dan Rahman, 2025). Hasil pengujian pada sistem tersebut menunjukkan tingkat deteksi kerusakan (*tamper detection*) sebesar 100% terhadap simulasi injeksi log palsu, serta pengurangan kebutuhan penyimpanan hingga 92% dibandingkan penyimpanan log mentah secara langsung pada *blockchain* (Islam dan Rahman, 2025).

BAB III

ANALISIS PERSOALAN DAN RANCANGAN SOLUSI

III.1 Analisis

III.1.1 Analisis Masalah

III.1.1.1 Analisis Sistem DecMed

Sebagaimana dijelaskan pada Subbab II.7, DecMed adalah kerangka kerja manajemen RME terdesentralisasi dengan menggunakan DLT IOTA sebagai penyimpanan on-chain, IPFS sebagai penyimpanan off-chain, serta PRE untuk mendukung distribusi akses data yang aman Purnama dkk., 2026. Arsitektur ini sudah memberikan fondasi untuk menjaga integritas data dan mengurangi ketergantungan pada penyimpanan terpusat, sebagaimana telah diuraikan pada Subbab II.5.1, II.5.2, dan II.6.1.

Meskipun arsitektur tersebut telah berhasil meningkatkan aspek keamanan penyimpanan data, DecMed lebih berfokus pada pencegahan serangan dan perlindungan data melalui kontrol akses. Aspek audit trail sebagai mekanisme untuk menghasilkan, menyimpan, dan mengelola bukti digital selama proses investigasi insiden keamanan masih belum menjadi fokus pembahasan, sebagaimana ditunjukkan pada Subbab II.7, di mana DecMed baru mengklaim ketersediaan audit trail secara umum tanpa merinci komponen-komponen audit trail yang telah diuraikan pada Subbab II.2.3.

Implementasi DecMed telah menyediakan beberapa bentuk pencatatan aktivitas sistem, yaitu *PatientAccessLog* dan *transaction block* pada jaringan IOTA. Kedua komponen tersebut telah mendokumentasikan sebagian aktivitas yang terjadi selama sistem beroperasi. Meskipun demikian, keduanya belum membentuk sebuah sistem *audit trail* yang mampu memenuhi kebutuhan audit maupun investigasi insiden keamanan, apabila dinilai terhadap karakteristik audit trail yang baik pada

Subbab II.2.2.

PatientAccessLog digunakan untuk mencatat aktivitas pemberian hak akses dari pasien kepada tenaga medis. Informasi yang tersimpan telah mencakup atribut penting seperti identitas pasien, identitas tenaga medis, waktu pemberian akses (*timestamp*), serta informasi lain yang berkaitan dengan hak akses tersebut. Dengan demikian, aktivitas pemberian hak akses telah terdokumentasi secara cukup rinci.

Namun demikian, *PatientAccessLog* masih memiliki beberapa keterbatasan. Pertama, cakupan aktivitas yang dicatat masih terbatas pada proses pemberian hak akses sehingga berbagai aktivitas lain yang memiliki nilai audit tinggi, seperti autentikasi, otorisasi, perubahan konfigurasi, maupun interaksi terhadap data rekam medis, belum terdokumentasi. Kondisi ini bertentangan dengan cakupan minimal peristiwa pemicu (*trigger events*) yang telah diuraikan pada Subbab II.2.3.1, yaitu peristiwa akses (*access events*) dan peristiwa kueri (*query events*) terhadap data kesehatan pribadi. Kedua, *PatientAccessLog* masih dikaitkan secara langsung dengan fungsionalitas pencabutan hak akses (*revoke access*). Ketika hak akses dicabut, informasi pada entri log yang sama akan diperbarui dengan mengubah status akses menjadi *revoked*. Pendekatan tersebut menyebabkan isi log dapat berubah setelah dibuat sehingga tidak memenuhi karakteristik integritas dan kerahasiaan audit trail yang telah diuraikan pada Subbab II.2.2, yang mensyaratkan bahwa catatan yang telah dibuat harus terlindungi dari upaya perusakan (*tampering*). Dalam konteks audit trail, setiap aktivitas yang telah terjadi seharusnya dipertahankan sebagai catatan historis yang tidak berubah, sedangkan perubahan kondisi akibat aktivitas berikutnya dicatat sebagai *audit record* baru yang terpisah, konsisten dengan perbedaan *audit record*, *audit log*, dan *audit trail* pada Subbab II.2.1. Akibatnya, aktivitas pemberian dan pencabutan hak akses tidak terdokumentasi sebagai dua peristiwa yang berdiri sendiri sehingga kronologi aktivitas menjadi kurang akurat.

Selain *PatientAccessLog*, DecMed juga memanfaatkan *transaction block* pada jaringan IOTA sebagai bukti transaksi yang terjadi di dalam sistem. Setiap pemanggilan fungsi *smart contract* akan menghasilkan sebuah *transaction block* yang tersimpan secara terdistribusi pada jaringan IOTA. Mekanisme ini memberikan jaminan integritas yang tinggi karena data transaksi bersifat *immutable* dan sulit dimodifikasi setelah berhasil dikonfirmasi oleh jaringan, sejalan dengan karakteristik *immutability* pada DLT yang telah diuraikan pada Subbab II.5.1.

Meskipun memiliki keunggulan pada aspek integritas, *transaction block* IOTA tidak dirancang sebagai media *audit trail*. Struktur data transaksi lebih berorientasi pada kebutuhan validasi transaksi pada jaringan DLT sehingga kurang mudah dipahami oleh auditor maupun administrator sistem. Selain itu, proses pencarian (*searching*) dan analisis transaksi juga kurang efisien karena tidak tersedia mekanisme *query*, indeks, maupun agregasi log yang memadai, sehingga belum memenuhi komponen pembacaan event yang telah diuraikan pada Subbab II.2.3.5. Kondisi tersebut menyulitkan proses investigasi ketika jumlah transaksi semakin besar. Di sisi lain, *transaction block* juga belum dapat menjamin retensi jangka panjang karena data transaksi pada node IOTA dapat dihapus melalui mekanisme *pruning*, sehingga belum memenuhi kebutuhan retensi pada komponen penyimpanan event yang telah diuraikan pada Subbab II.2.3.4.

Berdasarkan hasil analisis tersebut, dapat disimpulkan bahwa DecMed telah memiliki beberapa bentuk pencatatan aktivitas, namun belum memiliki sebuah sistem *audit trail* yang terintegrasi. Audit record masih tersebar pada beberapa komponen dengan karakteristik yang berbeda, belum tersedia mekanisme pengelolaan yang mampu menjaga integritas dan retensi audit trail, serta belum tersedia fasilitas yang mendukung proses pencarian maupun analisis audit trail secara efisien. Kondisi tersebut menyebabkan informasi yang tersedia belum dapat dimanfaatkan secara optimal sebagai bukti digital dalam proses audit maupun investigasi insiden keamanan.

Oleh karena itu, penelitian ini mengusulkan suatu sistem *audit trail* yang mampu menghasilkan *audit record* dari berbagai komponen sistem, mengelolanya secara terpusat, serta menyediakan mekanisme penyimpanan yang mampu menjaga integritas, mendukung retensi, dan mempermudah proses analisis sebagai bukti digital dalam investigasi insiden keamanan.

III.1.2 Dekomposisi Aplikasi

Dekomposisi aplikasi merupakan tahap awal proses *threat modeling* yang bertujuan memperoleh pemahaman menyeluruh mengenai sistem DecMed dan bagaimana sistem tersebut berinteraksi dengan entitas eksternal. Dekomposisi ini terdiri atas deskripsi sistem, dependensi eksternal, titik masuk, aset, dan level kepercayaan, yang selanjutnya menjadi dasar bagi proses identifikasi ancaman menggunakan

metode STRIDE.

III.1.2.1 Deskripsi Sistem

DecMed adalah kerangka kerja manajemen Rekam Medis Elektronik (RME) terdesentralisasi yang dibangun di atas Distributed Ledger Technology (DLT) IOTA. Sistem ini mengimplementasikan Capability-Based Access Control (CapBAC), Proxy Re-Encryption (PRE), dan InterPlanetary File System (IPFS) untuk menjamin kepemilikan data medis oleh pasien. Pasien secara aktif memberikan atau mencabut akses, menentukan durasi akses, dan berbagi data secara selektif dengan tenaga kesehatan. Sistem diimplementasikan menggunakan *smart contract* berbahasa Move pada ledger IOTA, sementara data klinis terenkripsi disimpan pada IPFS.

Arsitektur sistem terdiri atas tiga lapisan (*tiers*):

- **Client Layer** – antarmuka utama bagi seluruh aktor, dengan aplikasi *client* yang disesuaikan per peran (mobile untuk pasien, desktop untuk tenaga fasyankes dan Kementerian Kesehatan), dibangun menggunakan *framework* Tauri (Rust + SvelteKit/TypeScript).
- **Proxy Layer** – middleware keamanan yang menegakkan kebijakan akses dan menangani operasi kriptografis (PRE), terdiri atas PRE Server (Rust, Axum) dan Gas Station Server.
- **Storage Layer** – terbagi menjadi penyimpanan *on-chain* (IOTA ledger, untuk metadata dan *capability* akses) dan *off-chain* (IPFS untuk data klinis terenkripsi, Redis untuk data sementara seperti *nonce* dan *kfrag*).

Seluruh akses terhadap data RME pasien oleh pihak lain wajib melalui Proxy, dan setiap pemberian akses oleh pasien dicatat sebagai entri *capability* pada penyimpanan *on-chain* yang bersifat *immutable*, membentuk fondasi model CapBAC pada sistem DecMed.

III.1.2.2 Dependensi Eksternal

Dependensi eksternal merupakan objek di luar kode aplikasi yang keberadaannya dapat menimbulkan ancaman terhadap sistem. Setiap dependensi diberi nomor unik dan deskripsi.

Tabel III.1: Dependensi Eksternal Sistem DecMed

ID	Komponen	Deskripsi
DE-01	IOTA Rebased	DLT berbasis Directed Acyclic Graph (DAG) dengan konsensus Mysticti (Byzantine Fault Tolerant, Delegated Proof-of-Stake), finalitas transaksi ~500 ms, mendukung lebih dari 50.000 TPS.
DE-02	Smart Contract Move	<i>package</i> decmed yang dipublikasikan pada IOTA Layer 1, terdiri atas modul <i>patient</i> , <i>admin</i> , <i>hospital_personnel</i> , <i>proxy</i> , <i>shared</i> , serta modul struktur data <i>std_struct_*</i> dan <i>std_enum_*</i> .
DE-03	PRE Server	diimplementasikan dengan Rust dan <i>framework</i> Axum, menyediakan endpoint REST untuk proses autentikasi, pertukaran kunci, dan operasi RME.
DE-04	IOTA Gas Station	terdiri atas Gas Station Server (endpoint reservasi dan eksekusi <i>gas object</i>), Gas Station Storage (berbasis Redis dengan skrip LUA untuk akses konkuren), dan Key Store Manager/KSM (KMS eksternal atau penyimpanan kunci <i>in-memory</i>).
DE-05	Redis	digunakan oleh PRE Server untuk menyimpan data sementara seperti <i>nonce</i> autentikasi (TTL 3 menit) dan material PRE seperti <i>kfrag</i> (TTL 2 jam); juga digunakan oleh Gas Station sebagai <i>storage gas object</i> .
DE-06	IPFS Cluster	penyimpanan <i>off-chain</i> terdesentralisasi berbasis <i>content-addressing</i> (CID) untuk data klinis pasien yang telah dienkrupsi, digunakan karena ukuran data klinis dapat melebihi batas objek IOTA (250 KB).

ID	Komponen	Deskripsi
DE-07	Client Application Framework	Tauri (Rust + SvelteKit/TypeScript), dengan tiga varian: Patient Client (mobile), Hospital Client (desktop), dan Ministry Client (desktop).
DE-08	Skema BIP39	standar <i>mnemonic</i> 12 kata untuk pembuatan dan pemulihan kunci kriptografis pasien dan tenaga fasyankes secara <i>non-custodial</i> .
DE-09	Mekanisme kriptografis Proxy Re-Encryption (PRE) dan AES-GCM	digunakan untuk enkripsi data administratif dan klinis serta proses re-enkripsi <i>ciphertext</i> bagi penerima yang berwenang.

III.1.2.3 Titik Masuk

Titik masuk (*Entrypoints*) merupakan antarmuka pada aplikasi yang menjadi media interaksi antara pengguna (atau penyerang) dengan aplikasi maupun data. Setiap titik masuk diberi nomor unik, deskripsi, dan level kepercayaan minimum yang mengacu pada Tabel III.4.

Tabel III.2: Titik Masuk Sistem DecMed

ID	Nama	Deskripsi	Level Kepercayaan
TM-01	POST /nonce	Endpoint PRE Server untuk memperoleh <i>nonce</i> autentikasi 64-byte, divalidasi melalui <i>Move call is_account_registered</i> .	LK1–LK2

ID	Nama	Deskripsi	Level Kepercayaan
TM-02	POST /keys	Endpoint PRE Server untuk menyimpan material PRE (<i>kfrag</i> , kunci publik) dan menerbitkan token akses JWT bagi tenaga fasyan-kes.	LK2
TM-03	GET /medical-record	Endpoint PRE Server untuk mengambil metadata administratif dan klinis pasien beserta proses re-enkripsi, memerlukan token akses <i>read</i> yang valid.	LK4
TM-04	POST /medical-record	Endpoint PRE Server untuk membuat entri RME klinis baru, memerlukan token akses <i>update</i> .	LK4
TM-05	PUT /medical-record	Endpoint PRE Server untuk mengubah entri RME klinis yang sudah ada pada kunjungan yang sama.	LK4
TM-06	GET /medical-record/update	Endpoint PRE Server untuk mengambil metadata RME sebelum proses pengubahan (edit).	LK4
TM-07	GET /administrative	Endpoint PRE Server untuk mengambil metadata administratif pasien, dapat diakses oleh admin fasyankes maupun tenaga administratif.	LK3, LK5

ID	Nama	Deskripsi	Level Kepercayaan
TM-08	Modul decmed::patient	Kumpulan <i>method on-chain</i> yang dapat dieksekusi pasien: signup, create_access, revoke_access, update_administrative_metadata, get_medical_record(s), get_access_log, dan sejenisnya.	LK1–LK2
TM-09	Modul decmed::hospital_personnel	Kumpulan <i>method on-chain</i> bagi tenaga fasyankes: signup, use_activation_key, create_activation_key, cleanup_read/update_access, get_read/update_access, dan sejenisnya.	LK3–LK5
TM-10	Modul decmed::admin	Kumpulan <i>method on-chain</i> bagi Kementerian Kesehatan: create_activation_key (registrasi fasyankes), update_activation_key, get_hospitals.	LK6

ID	Nama	Deskripsi	Level Kepercayaan
TM-11	Modul decmed::proxy	Kumpulan <i>method on-chain</i> yang dieksekusi oleh PRE Server: <i>create_capability</i> , <i>create/update_medical_record</i> , <i>get_administrative_data</i> , <i>get_hospital_personnel_role</i> , dan sejenisnya.	LK7
TM-12	Gas Station endpoint (<i>reserve & execute</i>)	Dua endpoint utama Gas Station Server untuk men-cadangkan objek <i>gas</i> dan mengeksekusi transaksi ter-sponsori atas nama <i>client</i> .	LK7, LK8
TM-13	QR Code Delegasi Akses	Antarmuka fisik/visual berisi <i>iota_address</i> dan <i>pre_public_key</i> tenaga medis, dipindai oleh Patient Client untuk memulai proses delegasi akses.	LK2, LK4
TM-14	Sign-in lokal (PIN/biometrik)	Mekanisme autentikasi harian pasien/tenaga fasyan-kes menggunakan PIN atau biometrik terhadap kunci privat yang tersimpan ter-enkripsi di perangkat.	LK1–LK5

III.1.2.4 Aset

Aset merupakan hal fisik maupun abstrak yang dimiliki sistem dan menjadi target ketertarikan penyerang. Setiap aset diberi nomor unik, deskripsi, dan level kepercayaan yang mengacu pada Tabel III.4.

Tabel III.3: Aset Sistem DecMed

ID	Nama	Deskripsi
A-01	Data administratif RME	Identitas dan data sosial pasien (NIK, nama, dsb.), terenkripsi AES-GCM sebelum disimpan <i>on-chain</i> .
A-02	Data klinis RME	Data pemeriksaan, diagnosis, dan tindakan medis, terenkripsi AES-GCM dan disimpan pada IPFS sebagai <code>enc_medical_data</code> .
A-03	Entri <i>capability</i> akses (<i>on-chain</i>)	Struktur data <code>hospital_personnel_access_data</code> berisi <code>access_data_types</code> , <code>exp</code> (durasi kedaluwarsa), <code>metadata</code> , dan <code>medical_metadata_index</code> – fondasi model CapBAC.
A-04	<i>Recovery phrase</i> (BIP39) & kunci privat pasien/personel	<i>Mnemonic</i> 12 kata dan kunci privat turunannya, disimpan terenkripsi pada perangkat menggunakan <i>secure hardware</i> .
A-05	Material PRE (<code>kfrag</code> , <code>cfrag</code> , <i>pre_secret/public_key</i>)	Komponen kriptografis yang memungkinkan proses re-enkripsi data pasien kepada penerima yang berwenang.
A-06	<i>Nonce</i> autentikasi	Nilai acak 64-byte, TTL 3 menit, disimpan di Redis untuk mencegah serangan <i>replay</i> pada proses autentikasi pasien.
A-07	Token akses JWT	Token dengan klaim <i>role</i> dan <i>access type</i> (Read/Update); durasi berbeda antara tenaga medis (15 menit baca, 2 jam tulis) dan tenaga administratif.
A-08	<i>Activation key</i>	Kredensial sekali pakai yang diterbitkan Kementerian Kesehatan (untuk admin fasyankes) dan admin fasyankes (untuk personel), berfungsi sebagai <i>cryptographic witness</i> atas tindakan yang diotorisasi.

ID	Nama	Deskripsi
A-09	GlobalAdminCap & ProxyCapEnum	Representasi <i>capability</i> tertinggi pada sistem – masing-masing untuk Kementerian Kesehatan dan PRE Server; kompromi terhadap aset ini setara dengan eskalasi privilege total.
A-10	CID data klinis (IPFS)	<i>Content Identifier</i> yang menjadi referensi lokasi dan <i>fingerprint</i> integritas data klinis terenkripsi pada IPFS.
A-11	<i>Gas object</i> & kunci penandatanganan Gas Station	Token IOTA yang dipecah menjadi objek <i>gas</i> untuk mensponsori transaksi, beserta kunci penandatanganan yang dikelola KSM.
A-12	Log akses RME	Catatan riwayat akses terhadap data RME pasien, dapat dilihat pasien melalui <code>get_access_log</code> .

III.1.2.5 Level Kepercayaan

Level kepercayaan merepresentasikan hak akses yang diberikan sistem kepada suatu entitas eksternal maupun proses internal. Penentuan level kepercayaan pada sistem DecMed menggunakan pendekatan gabungan (*hybrid*) antara *role* untuk entitas eksternal (manusia) dan *capability* untuk proses/entitas internal, mengingat sistem DecMed secara eksplisit mengimplementasikan model Capability-Based Access Control (CapBAC).

Tabel III.4: Level Kepercayaan Sistem DecMed

ID	Nama	Deskripsi
LK1	Aktor belum teraktivasi/terregistrasi	Pengguna yang belum menyelesaikan proses <i>signup</i> atau aktivasi <i>client</i> ; tidak memiliki <i>capability</i> apa pun pada sistem.
LK2	Pasien (teraktivasi)	Pasien yang telah teregistrasi; memiliki <i>capability</i> membaca RME sendiri, memberi dan mencabut akses, serta melihat log akses.

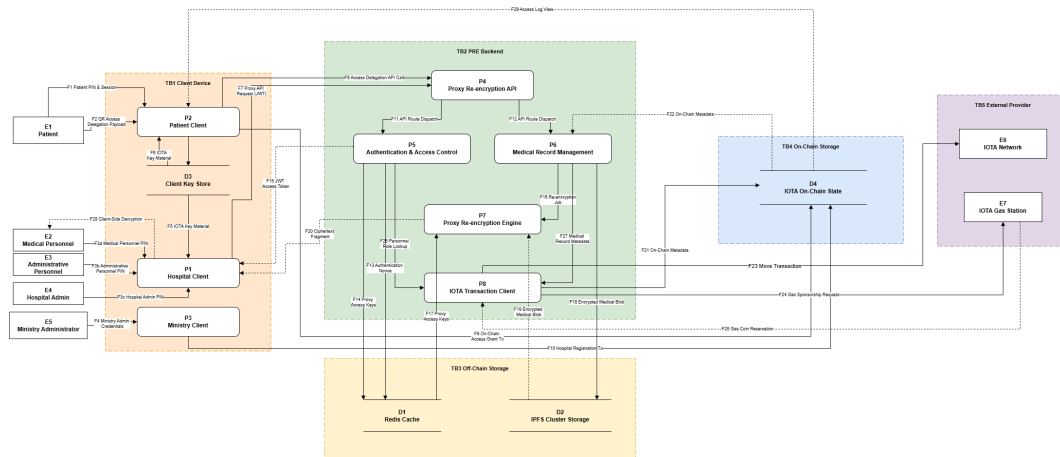
ID	Nama	Deskripsi
LK3	Administrative Personnel (dengan <i>capability</i> aktif)	Tenaga administratif fasyankes yang telah diberi akses oleh pasien; <i>scope</i> akses terbatas pada data administratif saja.
LK4	Medical Personnel (dengan <i>capability</i> aktif)	Tenaga medis fasyankes yang telah diberi akses oleh pasien; <i>scope</i> akses mencakup data administratif dan klinis, termasuk hak tulis (tambah/ubah) data klinis.
LK5	Hospital Admin	Administrator fasyankes; berwenang menambah personel dan menerbitkan <i>activation key</i> pada lingkup fasyankesnya.
LK6	Ministry of Health	Administrator Kementerian Kesehatan; berwenang meregistrasi fasyankes baru beserta adminnya, otoritas tertinggi di antara aktor manusia.
LK7	PRE Server (Proxy)	Proses <i>backend</i> dengan <i>capability</i> ProxyCapEnum; berwenang memverifikasi entri <i>capability on-chain</i> dan melakukan re-enkripsi data.
LK8	Gas Station Server	Proses <i>backend</i> yang mensponsori dan mengeksekusi transaksi atas nama <i>client</i> , memiliki akses terhadap kunci penandatanganan (KSM).
LK9	IOTA Network Validator	Node jaringan yang berpartisipasi dalam konsensus Mysticeti untuk memvalidasi dan memfinalisasi transaksi pada ledger IOTA.

Catatan: LK3 dan LK4 bersifat *parallel*, bukan hierarkis – keduanya diberikan *capability* oleh pasien secara independen dengan cakupan data yang berbeda (administratif saja vs. administratif dan klinis), sehingga tidak dapat diasumsikan LK4 selalu memiliki privilese lebih tinggi dari LK3 di seluruh konteks ancaman.

III.1.2.6 Data Flow Diagram

Data Flow Diagram (DFD) digunakan untuk menggambarkan aliran data antara entitas eksternal, proses, dan penyimpanan data pada sistem DecMed secara visual,

sebagai dasar identifikasi elemen yang akan diklasifikasikan ancamannya menggunakan metode STRIDE pada subbab berikutnya. DFD sistem DecMed digambarkan pada Gambar III.1.



Gambar III.1: Data Flow Diagram Sistem DecMed

DFD tersebut menggambarkan interaksi antara entitas eksternal (aktor), proses, dan penyimpanan data yang telah diidentifikasi pada dependensi eksternal (Tabel III.1), titik masuk (Tabel III.2), aset (Tabel III.3), dan level kepercayaan (Tabel III.4) pada subbab sebelumnya. Elemen-elemen pada DFD tersebut menjadi dasar penomoran dan referensi pada proses identifikasi ancaman menggunakan metode STRIDE (III.1.2.2).

III.1.3 Identifikasi Ancaman DecMed

Identifikasi ancaman dilakukan menggunakan pendekatan *STRIDE-per-Interaction*. Setiap interaksi antar elemen pada Data Flow Diagram (Gambar III.1) diperiksa berdasarkan arah dan tipe interaksinya (*process* ke *data store*, *process* ke *process*, *data flow* yang melintasi *machine boundary*, dsb.), kemudian kategori STRIDE yang dianalisis dibatasi hanya pada kategori yang relevan bagi tipe interaksi tersebut, mengacu pada Tabel II.2. Pendekatan ini memberikan justifikasi eksplisit atas kategori yang dipilih maupun yang tidak dianalisis untuk setiap ancaman.

Elemen E6 IOTA Network dan E7 IOTA Gas Station pada TB5 External Provider diperlakukan sebagai infrastruktur eksternal tepercaya yang berada di luar kendali sistem DecMed; ancaman terhadap operasional internal keduanya berada di luar cakupan *threat model* ini, dan hanya interaksinya dengan sistem (melalui P8, sebagai *process* yang mengirim output ke *external interactor*) yang dianalisis.

Hasil identifikasi ancaman disajikan pada Tabel III.5, dengan kolom **Tipe Interaksi** merujuk pada nomor baris interaksi di Tabel II.2 yang menjadi dasar penentuan kategori STRIDE bagi ancaman terkait.

Tabel III.5: Identifikasi Ancaman Sistem DecMed

ID	Deskripsi Ancaman	STRIDE
T1	Penyerang mendapatkan kredensial atau membajak sesi lokal milik pengguna untuk berpura-pura menjadi pemilik identitas sah saat <i>sign-in</i> .	S
T2	Kredensial <i>sign-in</i> (PIN) tersadap atau terekam melalui <i>shoulder surfing</i> , <i>keylogger</i> , atau malware pada perangkat saat dimasukkan ke Client.	I
T3	Percobaan <i>sign-in</i> berulang secara masif menghabiskan sumber daya Client (<i>local resource exhaustion</i>).	D
T4	Payload QR Access Delegation dipalsukan sebelum dipindai oleh Patient Client.	S
T5	Payload QR Access Delegation disadap sehingga informasi delegasi akses bocor kepada pihak tidak berwenang.	I
T6	Pasien menyangkal pernah memberikan akses (<i>create_access</i>) kepada tenaga medis tertentu.	R
T7	Tenaga medis atau tenaga administratif menyangkal pernah mengakses maupun mengubah data pasien tertentu.	R
T8	Admin fasyankes menyangkal pernah menerbitkan <i>activation key</i> kepada personel tertentu.	R
T9	Ministry Administrator menyangkal pernah meregistrasi fasyankes tertentu beserta admin-nya.	R
T10	Penyerang menyamar sebagai Proxy Re-encryption API (P4) yang sah (<i>man-in-the-middle</i>) saat Client mengirim permintaan.	S
T11	Penyerang menyadap komunikasi Client dengan PRE Server, mengungkap data sensitif seperti token JWT maupun data administratif.	I

ID	Deskripsi Ancaman	STRIDE
T12	Pihak eksternal membanjiri <i>endpoint</i> dengan permintaan palsu untuk menghabiskan sumber daya PRE Server.	D
T13	Token JWT dipalsukan atau diputar ulang (<i>replay</i>) pada permintaan dari Client ke PRE Server.	S
T14	Penyerang memodifikasi <i>Move transaction</i> pada perjalanan dari IOTA Transaction Client menuju IOTA Network.	T
T15	Penyerang menyamar sebagai IOTA Transaction Client (P8) yang sah saat berinteraksi dengan Gas Station untuk mengajukan sponsorship transaksi palsu.	S
T16	PRE Server (P8) menyangkal telah mengajukan atau menerima keputusan sponsorship transaksi tertentu dari Gas Station.	R
T17	Penyerang mengajukan permintaan sponsorship <i>gas</i> secara berulang sehingga <i>gas object</i> habis dan transaksi sah pengguna lain gagal tersponsori.	D
T18	Pihak dengan akses tidak sah ke Redis memodifikasi nilai <i>nonce</i> autentikasi atau <i>kfrag</i> sebelum digunakan.	T
T19	Pihak dengan akses tidak sah ke Redis membaca <i>kfrag</i> atau <i>nonce</i> sebelum masa berlakunya (TTL) berakhir.	I
T20	Redis dipenuhi (<i>memory exhaustion</i>) melalui permintaan <i>nonce</i> baru secara berulang sehingga proses autentikasi pengguna sah terganggu.	D
T21	Data klinis terenkripsi diunduh dari IPFS Cluster Storage oleh pihak tidak berwenang.	I
T22	IPFS Cluster dibanjiri permintaan berukuran besar sehingga mengganggu ketersediaan data klinis bagi pengguna sah.	D
T23	Metadata <i>on-chain</i> yang tersimpan pada IOTA On-Chain State bersifat publik sehingga pola transaksi berpotensi dianalisis untuk mengungkap informasi tidak langsung mengenai pasien.	I

ID	Deskripsi Ancaman	STRIDE
T24	Kelemahan validasi <i>capability on-chain</i> memungkinkan pembacaan atau pembuatan ProxyCapEnum/GlobalAdminCap tanpa otorisasi sah, setara privilege tertinggi PRE Server/Kementerian Kesehatan.	E
T25	Kegagalan validasi otorisasi pada Authentication & Access Control memungkinkan Medical Record Management memproses permintaan dengan <i>scope</i> lebih tinggi dari yang seharusnya.	E

III.1.4 Analisis Kebutuhan Audit Event

Berdasarkan hasil analisis ancaman menggunakan metode STRIDE pada bagian sebelumnya, tahap selanjutnya adalah menentukan kebutuhan audit trail yang diperlukan untuk menghasilkan bukti digital terhadap setiap ancaman yang teridentifikasi. Kebutuhan tersebut diperoleh dengan memetakan setiap ancaman ke aktivitas sistem yang perlu dicatat sebagai audit event. Hasil analisis ini menjadi dasar dalam mengidentifikasi kesenjangan mekanisme audit pada DecMed serta menjadi masukan bagi perancangan sistem audit trail pada bab selanjutnya. Hasil pemetaan antara ancaman, kebutuhan bukti digital, dan audit event yang dihasilkan ditunjukkan pada Tabel III.6.

Tabel III.6: Pemetaan Ancaman ke Audit Event

Threat	Bukti yang Dibutuhkan	Audit Event
T1, T2, T3	Identitas pengguna, metode autentikasi, hasil autentikasi, waktu, perangkat, serta frekuensi percobaan autentikasi.	Authentication
T4, T5	Payload QR Access Delegation, hasil validasi, identitas pembuat dan pemindai QR.	QR Access Delegation Validation

Threat	Bukti yang Dibutuhkan	Audit Event
T6	Identitas pasien, penerima akses, <i>capability</i> yang dibuat, waktu, serta <i>transaction digest</i> .	Capability Creation
T7, T13	Identitas pengguna, jenis operasi (<i>read/write</i>), objek rekam medis, <i>capability</i> yang digunakan, serta hasil validasi otorisasi.	Medical Record Access
T8	Identitas Admin Fasyankes, identitas personel yang terkait, activation key dan waktu.	Hospital Personnel Activation Key Generation
T9	Identitas Ministry Administrator, identitas fasyankes yang diregistrasi, dan waktu.	Healthcare Facility Registration
T10, T11, T12	Identitas pemohon, layanan PRE yang diminta, endpoint tujuan, waktu, dan hasil pemrosesan permintaan.	PRE Service Request
T14	<i>Transaction digest</i> , identitas penandatanganan, payload transaksi, dan hasil pengiriman transaksi.	IOTA Transaction Submission
T15, T16, T17	Identitas pemohon sponsorship, <i>transaction digest</i> , dan waktu.	Gas Sponsorship Request
T18, T19, T20	Objek yang diakses, jenis operasi, identitas proses, dan waktu operasi.	Redis Operation

Threat	Bukti yang Dibutuhkan	Audit Event
T21, T22	Content Identifier (CID), identitas pemohon, jenis operasi, dan waktu akses.	IPFS Object Access
T23	Objek <i>on-chain</i> yang diakses, identitas pemohon, dan waktu akses.	On-chain Metadata Access
T25, T24	Capability ID, identitas pemohon, hasil validasi otorisasi, serta alasan penolakan apabila validasi gagal.	Capability Validation

III.1.4.1 Identifikasi Atribut Audit

Setelah diperoleh daftar *audit event* pada Tabel III.6, tahap selanjutnya adalah mengidentifikasi atribut yang perlu dicatat setiap kali *audit event* tersebut terjadi. Kumpulan atribut tersebut membentuk sebuah *audit record* yang menjadi bukti digital atas aktivitas sistem. Penentuan atribut audit dilakukan dengan mengacu pada kontrol AU-3 *Content of Audit Records* pada NIST SP 800-53 Rev. 5 National Institute of Standards and Technology, 2020, yang menyatakan bahwa setiap *audit record* harus memuat informasi mengenai (a) jenis *event* yang terjadi, (b) waktu kejadian, (c) lokasi atau sumber kejadian, (d) sumber *event*, (e) hasil (*outcome*) dari *event*, serta (f) identitas individu, subjek, maupun objek yang terlibat.

Keenam elemen tersebut kemudian diimplementasikan dalam bentuk atribut audit umum yang dicatat pada seluruh *audit event* sebagaimana ditunjukkan pada Tabel III.7. Selain atribut umum tersebut, setiap *audit event* juga memiliki atribut tambahan yang disesuaikan dengan karakteristik aktivitas sistem dan kebutuhan bukti digital terhadap ancaman yang dipetakan sebelumnya. Atribut tambahan tersebut disajikan pada Tabel III.8.

Field *record_id* dan *prev_record_hash* tidak secara eksplisit disyaratkan oleh kontrol AU-3. Kedua atribut tersebut ditambahkan untuk memenuhi kebutuhan rancangan *audit trail* pada penelitian ini, khususnya dalam menjamin identitas unik

Tabel III.7: Skema Umum Atribut Audit

Atribut	Deskripsi	Elemen AU-3
record_id	Pengenal unik setiap <i>audit record</i> (UUID) yang tidak dapat digunakan ulang.	–
event_type	Jenis <i>audit event</i> yang terjadi, mengacu pada daftar <i>audit event</i> pada Tabel ??.	(a)
timestamp	Waktu terjadinya <i>audit event</i> dengan presisi milidetik dalam format UTC.	(b)
source_component	Komponen atau proses sistem yang menghasilkan <i>audit event</i> .	(c), (d)
actor_id	Identitas aktor yang memicu <i>audit event</i> , seperti <i>IOTA address</i> , <i>role</i> , atau <i>session ID</i> .	(f)
target_object	Identitas objek atau sumber daya yang menjadi target aktivitas, misalnya ID rekam medis, ID <i>capability</i> , atau CID IPFS.	(f)
outcome	Hasil pelaksanaan <i>audit event</i> , seperti <i>success</i> , <i>failure</i> , atau <i>denied</i> , beserta kode alasan apabila tersedia.	(e)
prev_record_hash	<i>Hash</i> dari <i>audit record</i> sebelumnya untuk membentuk mekanisme <i>hash chaining</i> yang mendukung sifat <i>append-only</i> dan <i>immutable</i> .	–

setiap *audit record* serta menjaga integritas rangkaian *audit record* melalui mekanisme *hash chaining*.

Tabel III.8: Atribut Audit Tambahan untuk Setiap Audit Event

ID Event	Atribut Tambahan	Alasan
EV1	auth_method, authentication_result, failed_attempt_count, device_fingerprint.	Mendukung identifikasi metode autentikasi, pola kegagalan autentikasi, serta anomali perangkat yang digunakan.
EV2	qr_payload_id, recipient_identity, signature_valid.	Membuktikan keaslian payload QR serta identitas pihak yang menerima delegasi akses.

ID Event	Atribut Tambahan	Alasan
EV3	capability_id, access_scope, expiry_duration, transaction_digest.	Mendokumentasikan capability yang diberikan beserta ruang lingkup akses dan transaksi yang merekamnya.
EV4	access_type, medical_record_id, capability_id, authorization_token_id.	Mengidentifikasi jenis akses terhadap rekam medis beserta capability dan token otorisasi yang digunakan.
EV5	facility_id, facility_name, administrator_id, transaction_digest.	Mendokumentasikan registrasi fasilitas pelayanan kesehatan beserta transaksi yang merekamnya.
EV6	endpoint_called, request_id, caller_component, channel_encryption.	Mengidentifikasi layanan PRE yang dipanggil beserta komponen pemanggil dan status pengamanan kanal komunikasi.
EV7	reencryption_operation_id, capability_id, target_ciphertext, kfrag_identifier.	Mendokumentasikan proses re-enkripsi yang dilakukan terhadap data terenkripsi.
EV8	transaction_digest, payload_hash, network_confirmation_status.	Menyediakan bukti pengiriman transaksi beserta status konfirmasi pada jaringan IOTA.
EV9	requested_gas_budget, requester_id, transaction_digest.	Mendokumentasikan permintaan sponsorship beserta kebutuhan gas transaksi.
EV10	approval_status, decision_reason, approved_by.	Mencatat keputusan sponsorship beserta alasan keputusan dan pihak yang memberikan persetujuan.

ID Event	Atribut Tambahan	Alasan
EV11	key_operation, key_id, key_type.	Mencatat aktivitas pembuatan, perubahan, maupun penghapusan material kriptografi.
EV12	key_id, key_type, access_purpose.	Mencatat aktivitas akses terhadap material kriptografi beserta tujuan penggunaannya.
EV13	redis_key_type, operation_type, ttl_remaining.	Mencatat aktivitas penyimpanan atau perubahan objek sensitif pada Redis.
EV14	redis_key_type, request_origin, ttl_remaining.	Mencatat aktivitas pembacaan objek sensitif beserta asal permintaan.
EV15	cid, operation_type, data_size.	Mencatat aktivitas unggah maupun unduh objek pada penyimpanan IPFS.
EV16	cid, expected_hash, verification_result.	Mendokumentasikan proses verifikasi integritas objek pada IPFS.
EV17	metadata_type, object_id, query_requester_id.	Mencatat aktivitas pembacaan metadata atau objek pada ledger IOTA.
EV18	capability_checked, required_scope, actual_scope, validation_result.	Mendokumentasikan proses validasi capability beserta hasil evaluasi hak akses.

III.1.5 Analisis Kebutuhan Sistem

III.1.5.1 Analisis Komponen Infrastruktur Audit Trail

Setelah kebutuhan *audit event* ditentukan, langkah selanjutnya adalah mengidentifikasi komponen infrastruktur yang diperlukan agar proses pencatatan, penyimpanan,

pemantauan, dan analisis *audit trail* dapat berjalan secara menyeluruh. Analisis ini dilakukan dengan mengacu pada model konseptual sistem *audit trail* yang dijelaskan pada Annex B ISO 27789. Berbeda dengan NIST SP 800-53 yang mendefinisikan kontrol keamanan yang harus dipenuhi oleh suatu sistem *audit trail*, ISO 27789 menyediakan model layanan (*service-oriented architecture*) yang menggambarkan komponen-komponen utama beserta interaksinya dalam membangun sebuah sistem *audit trail*.

Model tersebut menunjukkan bahwa sistem *audit trail* tidak hanya terdiri atas media penyimpanan log, tetapi merupakan sekumpulan layanan yang bekerja sama untuk menerima *audit event*, membentuk *audit record*, menyimpan data, melakukan pemantauan terhadap aktivitas yang terjadi, hingga menyediakan fasilitas pencarian dan pelaporan bagi auditor. Berdasarkan model tersebut, komponen-komponen yang menjadi kebutuhan dasar infrastruktur *audit trail* dapat diidentifikasi sebagaimana ditunjukkan pada Tabel III.9.

Tabel III.9: Identifikasi Komponen Infrastruktur Audit Trail berdasarkan ISO 27789 Annex B

Komponen	Fungsi
Audit Logger	Menerima <i>audit event</i> dari aplikasi atau sistem, melakukan validasi awal, kemudian meneruskan <i>event</i> untuk diproses dan disimpan sebagai <i>audit record</i> .
Audit Record Generator	Membentuk <i>audit record</i> berdasarkan jenis <i>audit event</i> yang diterima sehingga seluruh <i>audit record</i> memiliki format yang konsisten.
Audit Event Catalogue	Menyediakan definisi, metadata, dan struktur setiap jenis <i>audit event</i> yang digunakan oleh sistem.
Audit Repository	Menyimpan seluruh <i>audit record</i> yang dihasilkan sehingga dapat digunakan sebagai bukti pada proses audit maupun investigasi.
Audit Monitor	Melakukan pemantauan terhadap aliran <i>audit event</i> untuk mendeteksi pola atau kondisi tertentu yang memerlukan perhatian lebih lanjut.

Komponen	Fungsi
Audit Alert	Menghasilkan notifikasi apabila Audit Monitor mendeteksi pola aktivitas yang memenuhi kondisi tertentu sesuai kebijakan yang ditetapkan.
Audit Report	Menyediakan fasilitas pencarian, pengambilan, dan penyajian <i>audit record</i> untuk mendukung proses audit dan investigasi.

Berdasarkan identifikasi tersebut, dapat dilihat bahwa sistem *audit trail* menurut ISO 27789 terdiri atas beberapa layanan yang memiliki tanggung jawab berbeda namun saling melengkapi. Audit Logger Service berperan sebagai titik masuk seluruh *audit event*, sedangkan Audit Repository bertanggung jawab sebagai media penyimpanan utama. Di sisi lain, Audit Report Service menyediakan akses terhadap data audit untuk keperluan investigasi, sementara Audit Monitor Service dan Audit Alert Service mendukung kemampuan pemantauan secara *real-time* terhadap aktivitas yang dicatat. Selain itu, Audit Event Catalogue Service dan Audit Record Generator Service berfungsi menjaga konsistensi struktur *audit record* yang dihasilkan oleh berbagai aplikasi.

Komponen-komponen tersebut selanjutnya akan dibandingkan dengan kondisi eksisting pada arsitektur DecMed untuk mengidentifikasi komponen yang telah tersedia maupun komponen yang masih belum tersedia. Hasil analisis tersebut menjadi dasar dalam menentukan kebutuhan infrastruktur *audit trail* yang akan dirancang pada penelitian ini.

III.1.5.2 Analisis Gap Infrastruktur Audit Trail terhadap DecMed

Setelah komponen-komponen utama infrastruktur *audit trail* diidentifikasi berdasarkan model konseptual ISO 27789, langkah selanjutnya adalah membandingkan model tersebut dengan kondisi eksisting sistem DecMed. Analisis ini bertujuan untuk mengidentifikasi komponen infrastruktur *audit trail* yang telah tersedia maupun komponen yang masih belum tersedia pada arsitektur DecMed. Hasil analisis tersebut digunakan untuk menentukan kebutuhan implementasi yang harus dipenuhi agar DecMed mampu mendukung proses pencatatan, penyimpanan, pemantauan, dan penyajian *audit trail* secara komprehensif.

Analisis dilakukan dengan mengevaluasi setiap komponen yang direkomendasikan oleh ISO 27789 terhadap arsitektur DecMed yang terdiri atas aplikasi klien, *smart contract* IOTA Move, *Proxy Re-Encryption Server*, serta infrastruktur penyimpanan data yang telah ada. Hasil analisis ditunjukkan pada Tabel III.10.

Tabel III.10: Analisis Gap Infrastruktur Audit Trail terhadap DecMed

Komponen	Kondisi DecMed	Analisis Gap	Kebutuhan
Audit Logger Service	Belum tersedia. Audit event dicatat secara terpisah oleh masing-masing komponen.	Belum terdapat mekanisme terpusat untuk menerima seluruh audit event dari berbagai komponen sistem.	Diperlukan layanan yang bertugas menerima seluruh audit event sebelum diproses lebih lanjut.
Audit Record Generator Service	Belum tersedia.	Belum terdapat mekanisme pembentukan audit record yang konsisten sehingga struktur audit record bergantung pada implementasi masing-masing komponen.	Diperlukan mekanisme pembentukan audit record yang memiliki format dan atribut yang seragam.
Audit Event Catalogue Service	Belum tersedia.	Belum terdapat definisi terpusat mengenai jenis audit event beserta atribut yang dimiliki.	Diperlukan katalog audit event sebagai acuan seluruh komponen dalam menghasilkan audit record.

Komponen	Kondisi DecMed	Analisis Gap	Kebutuhan
Audit Repository	Belum tersedia.	DecMed belum memiliki repositori khusus untuk menyimpan audit trail secara terpusat. Riwayat akses pasien dan transaksi blockchain belum dapat berfungsi sebagai repositori audit trail yang komprehensif.	Diperlukan repositori khusus yang mampu menyimpan audit trail secara terpusat dan mendukung kebutuhan investigasi.
Audit Monitor Service	Belum tersedia.	Belum terdapat mekanisme pemantauan terhadap pola aktivitas atau kejadian mencurigakan secara otomatis.	Diperlukan mekanisme monitoring audit event sebagai dasar deteksi aktivitas tertentu.
Audit Alert Service	Belum tersedia.	Belum terdapat mekanisme pemberian notifikasi ketika ditemukan aktivitas yang memenuhi kondisi tertentu.	Diperlukan layanan yang mampu menghasilkan notifikasi berdasarkan hasil pemantauan audit trail.

Komponen	Kondisi DecMed	Analisis Gap	Kebutuhan
Audit Report Service	Sebagian tersedia melalui <i>Patient Access Log</i> .	Riwayat akses yang tersedia hanya mencakup aktivitas tertentu dan belum mendukung pencarian maupun pelaporan audit trail secara menyeluruh.	Diperlukan layanan pelaporan yang mampu melakukan pencarian dan penyajian audit trail untuk proses audit maupun investigasi.

Berdasarkan Tabel III.10, dapat dilihat bahwa sebagian besar komponen infrastruktur *audit trail* yang direkomendasikan oleh ISO 27789 belum tersedia pada arsitektur DecMed. Meskipun DecMed telah memiliki beberapa mekanisme pencatatan aktivitas, seperti *Patient Access Log* dan riwayat transaksi pada IOTA DLT, mekanisme tersebut belum membentuk sebuah sistem *audit trail* yang terintegrasi. Selain itu, belum tersedia layanan yang secara khusus bertanggung jawab untuk menerima *audit event*, membentuk *audit record*, menyimpan data audit secara terpusat, maupun menyediakan fasilitas pemantauan dan pelaporan.

Hasil analisis ini menunjukkan bahwa pengembangan sistem *audit trail* pada DecMed tidak hanya memerlukan penambahan media penyimpanan audit trail, tetapi juga memerlukan beberapa komponen layanan baru yang bekerja secara terintegrasi. Komponen-komponen tersebut selanjutnya akan dianalisis berdasarkan kontrol keamanan yang direkomendasikan oleh NIST SP 800-53 Revision 5 untuk menentukan kebutuhan fungsional dan non-fungsional yang harus dipenuhi oleh setiap komponen sebelum dilakukan proses perancangan sistem.

III.1.5.3 Analisis Kebutuhan Keamanan Infrastruktur Audit Trail

Selain kebutuhan terhadap komponen infrastruktur, sistem *audit trail* juga harus memenuhi kebutuhan keamanan agar informasi audit yang dihasilkan dapat digunakan sebagai bukti yang valid dalam proses audit maupun investigasi. Untuk mengidentifikasi kebutuhan tersebut, penelitian ini mengacu pada keluarga kontrol *Audit and*

Accountability (AU) pada NIST SP 800-53 Revision 5. Kontrol-kontrol tersebut dianalisis untuk menentukan kebutuhan fungsional maupun non-fungsional yang harus dipenuhi oleh setiap komponen infrastruktur *audit trail* yang telah diidentifikasi sebelumnya.

Tidak seluruh kontrol pada keluarga AU dianalisis dalam penelitian ini. Analisis difokuskan pada kontrol yang memiliki keterkaitan langsung dengan perancangan infrastruktur *audit trail* pada DecMed. Hasil analisis kebutuhan keamanan berdasarkan kontrol NIST SP 800-53 Revision 5 ditunjukkan pada Tabel III.11.

Tabel III.11: Analisis Kebutuhan Infrastruktur Audit Trail berdasarkan NIST SP 800-53 Rev. 5

Kontrol	Nama Kontrol	Analisis	Kebutuhan Sistem
AU-2	Event Logging	Sistem harus menentukan aktivitas yang perlu dicatat sebagai audit trail berdasarkan kebutuhan organisasi dan risiko keamanan.	Sistem harus mampu mencatat seluruh audit event yang telah diidentifikasi pada tahap analisis audit event.
AU-3	Content of Audit Records	Setiap audit record harus memiliki informasi yang cukup untuk mendukung proses identifikasi dan investigasi suatu aktivitas.	Sistem harus menghasilkan audit record yang memiliki atribut lengkap seperti identitas pelaku, waktu kejadian, jenis aktivitas, objek yang diakses, dan hasil aktivitas.

Kontrol	Nama Kontrol	Analisis	Kebutuhan Sistem
AU-4	Audit Log Storage Capacity	Sistem harus menyediakan media penyimpanan yang memadai sehingga audit trail tetap tersedia selama periode retensi yang ditentukan.	Sistem harus menyediakan repositori audit yang mampu menyimpan audit trail sesuai kebutuhan retensi.
AU-5	Response to Audit Logging Process Failures	Kegagalan proses pencatatan audit harus dapat dideteksi dan ditangani agar tidak menyebabkan hilangnya audit trail.	Sistem harus mampu mendeteksi serta menangani kegagalan proses pencatatan audit.
AU-6	Audit Record Review, Analysis, and Reporting	Audit trail harus dapat ditinjau, dianalisis, dan disajikan untuk mendukung proses audit maupun investigasi.	Sistem harus menyediakan mekanisme pencarian, analisis, dan pelaporan audit trail.
AU-8	Time Stamps	Seluruh audit record harus menggunakan informasi waktu yang akurat dan konsisten.	Sistem harus memberikan timestamp yang konsisten pada setiap audit record.
AU-9	Protection of Audit Information	Audit trail harus terlindungi dari modifikasi maupun penghapusan yang tidak sah.	Sistem harus menjamin integritas dan perlindungan audit trail terhadap perubahan yang tidak sah.

Kontrol	Nama Kontrol	Analisis	Kebutuhan Sistem
AU-10	Non-repudiation	Audit trail harus mampu mendukung pembuktian bahwa suatu aktivitas benar-benar dilakukan oleh entitas tertentu.	Sistem harus mendukung non-repudiation melalui mekanisme yang menjaga keaslian audit record.
AU-11	Audit Record Retention	Audit trail harus dipertahankan selama periode yang telah ditentukan agar tetap tersedia untuk kebutuhan audit maupun investigasi.	Sistem harus menyediakan mekanisme retensi audit trail sesuai kebijakan penyimpanan.
AU-12	Audit Record Generation	Sistem harus secara otomatis menghasilkan audit record ketika audit event terjadi.	Sistem harus mampu menghasilkan audit record secara otomatis dari seluruh audit event yang telah ditentukan.
AU-13	Monitoring for Information Disclosure	Audit trail harus mendukung pemantauan terhadap aktivitas yang berpotensi menyebabkan pengungkapan informasi yang tidak sah.	Sistem harus mendukung pemantauan audit trail untuk mendeteksi aktivitas yang mencurigakan.

Kontrol	Nama Kontrol	Analisis	Kebutuhan Sistem
AU-14	Session Audit	Apabila diperlukan, sistem harus mampu menghubungkan audit trail dengan suatu sesi pengguna.	Sistem harus mendukung pencatatan informasi sesi untuk aktivitas yang memerlukan keterkaitan antar audit event.

Berdasarkan hasil analisis pada Tabel III.11, dapat diidentifikasi bahwa kebutuhan keamanan sistem *audit trail* tidak hanya berfokus pada kemampuan mencatat aktivitas, tetapi juga mencakup aspek integritas, ketersediaan, perlindungan, serta kemampuan audit trail dalam mendukung proses investigasi. Kontrol-kontrol tersebut kemudian diterjemahkan menjadi kebutuhan sistem yang harus dipenuhi oleh komponen-komponen infrastruktur *audit trail* yang telah diidentifikasi sebelumnya.

Hasil analisis ini menjadi dasar dalam penyusunan kebutuhan fungsional dan non-fungsional sistem *audit trail*. Kebutuhan tersebut selanjutnya digunakan sebagai acuan pada tahap perancangan arsitektur dan implementasi sistem yang diusulkan.

III.1.5.4 Ringkasan Kebutuhan Sistem Audit Trail

Berdasarkan hasil analisis yang telah dilakukan pada subbab sebelumnya, diperoleh kebutuhan sistem *audit trail* yang akan digunakan sebagai dasar dalam proses perancangan sistem. Kebutuhan tersebut disusun berdasarkan hasil identifikasi *audit event*, analisis komponen infrastruktur berdasarkan ISO 27789, analisis kesenjangan terhadap arsitektur DecMed, serta analisis kebutuhan keamanan berdasarkan keluarga kontrol *Audit and Accountability* pada NIST SP 800-53 Revision 5.

Kebutuhan sistem dibagi menjadi dua kategori, yaitu kebutuhan fungsional dan kebutuhan non-fungsional. Kebutuhan fungsional mendeskripsikan kemampuan yang harus dimiliki oleh sistem *audit trail*, sedangkan kebutuhan non-fungsional menjelaskan karakteristik kualitas dan keamanan yang harus dipenuhi oleh sistem.

Tabel III.12: Kebutuhan Fungsional Sistem Audit Trail

ID	Kebutuhan
FR-01	Sistem harus mampu menerima seluruh <i>audit event</i> yang dihasilkan oleh komponen-komponen DecMed sesuai dengan daftar <i>audit event</i> yang telah ditentukan.
FR-02	Sistem harus secara otomatis membangkitkan <i>audit record</i> setiap kali terjadi <i>audit event</i> .
FR-03	Setiap <i>audit record</i> harus memiliki struktur dan atribut yang konsisten sesuai dengan skema audit yang telah ditentukan.
FR-04	Sistem harus menyimpan seluruh <i>audit record</i> pada repositori audit yang terdedikasi.
FR-05	Sistem harus menyediakan mekanisme untuk mengambil audit record berdasarkan parameter tertentu sebagai kebutuhan proses audit dan investigasi.
FR-06	Sistem harus mampu mendeteksi kegagalan pada proses pembangunan maupun penyimpanan <i>audit record</i> .

Tabel III.13: Kebutuhan Non-Fungsional Sistem Audit Trail

ID	Kebutuhan
NFR-01	Sistem harus menjamin integritas setiap <i>audit record</i> sehingga tidak dapat dimodifikasi secara tidak sah.
NFR-02	Sistem harus menjamin keaslian (<i>authenticity</i>) setiap <i>audit record</i> .
NFR-03	Sistem harus memberikan <i>timestamp</i> yang akurat dan konsisten pada setiap <i>audit record</i> .
NFR-04	Sistem harus menerapkan mekanisme <i>append-only</i> sehingga <i>audit record</i> yang telah tersimpan tidak dapat diubah ataupun dihapus.
NFR-05	Sistem harus mempertahankan <i>audit record</i> selama periode retensi yang ditentukan.
NFR-06	Sistem harus mendukung <i>non-repudiation</i> sehingga <i>audit record</i> dapat digunakan sebagai bukti bahwa suatu aktivitas benar-benar dilakukan oleh entitas yang tercatat.

Kebutuhan fungsional dan non-fungsional yang telah dirumuskan menjadi acuan dalam tahap perancangan sistem *audit trail*. Pada tahap selanjutnya, setiap kebutuhan tersebut dipetakan ke dalam rancangan arsitektur, komponen, serta mekanisme implementasi yang sesuai dengan karakteristik arsitektur DecMed. Dengan demikian, rancangan sistem yang dihasilkan tidak hanya memenuhi kebutuhan fungsional pencatatan *audit trail*, tetapi juga memenuhi kebutuhan keamanan, integritas, dan akuntabilitas sebagaimana direkomendasikan oleh standar ISO 27789 dan NIST SP 800-53 Revision 5.

III.2 Rancangan

III.2.1 Arsitektur Sistem Audit Trail

Untuk mengatasi permasalahan yang telah diidentifikasi pada Analisis Masalah penelitian ini mengusulkan penambahan satu komponen baru pada arsitektur DecMed, yaitu **Audit Trail Service (ATS)**. ATS dirancang sebagai layanan *backend* berbasis Rust dan *framework* Axum yang independen terhadap komponen DecMed yang telah ada (PRE Server, Gas Station Server, maupun *smart contract*), sehingga penambahan mekanisme audit trail tidak mengubah alur bisnis maupun fungsionalitas DecMed yang sudah berjalan.

ATS terdiri atas tiga komponen fungsional utama:

- **Audit Receiver** yang menyediakan *endpoint* POST `/api/events` sebagai titik masuk tunggal seluruh audit *event* dari berbagai sumber pada sistem DecMed (Client Apps, PRE Server, Gas Station Server). Setiap *event* dikirimkan dalam format *signed payload* yang berisi tanda tangan digital Ed25519, memungkinkan ATS memverifikasi keaslian pengirim sebelum *event* diproses lebih lanjut.
- **Audit Logger** yang menerima *event* yang telah diverifikasi dari Audit Receiver melalui antrian asinkron internal (*mpsc channel*), membentuk *audit record* dengan atribut standar, dan menerapkan mekanisme *hash-chaining* antar *record* sehingga setiap *record* menyimpan `prev_record_hash` dari *record* sebelumnya. Mekanisme ini menjamin bahwa modifikasi atau penghapusan satu *record* akan terdeteksi karena seluruh *record* sesudahnya akan memiliki

nilai `record_hash` yang tidak konsisten.

- **Log Rotation & Archival Worker** yang berjalan secara periodik dan bertanggung jawab atas tiga tahap pengarsipan, yaitu: (i) merotasi berkas log aktif menjadi berkas arsip baru, (ii) mengunggah berkas arsip ke IPFS Cluster yang telah digunakan DecMed sebagai penyimpanan *off-chain*, sehingga menghasilkan *Content Identifier* (CID) sebagai penanda integritas berkas, dan (iii) menerbitkan metadata rotasi log ke jaringan IOTA melalui Gas Station Server yang telah ada, sebagai *trust anchor* yang *immutable*.

Dengan arsitektur tersebut, ATS tidak menambah infrastruktur penyimpanan baru, melainkan memanfaatkan kembali (*reuse*) infrastruktur *off-chain* (IPFS Cluster) dan *on-chain* (IOTA via Gas Station Server) yang sudah tersedia pada DecMed.

III.2.2 Mekanisme Pembangkitan dan Penerimaan Audit Event

Audit *event* dibangkitkan dari tiga sumber yang telah diidentifikasi pada tahap analisis kebutuhan, yaitu Client Apps, PRE Server, dan Gas Station Server. Setiap sumber mengirimkan *event* ke ATS dalam format SignedEvent yang terdiri atas tiga bidang:

- payload berupa representasi JSON dari *audit event* yang berisi seluruh atribut kontekstual aktivitas yang dicatat,
- signature berupa tanda tangan digital Ed25519 atas payload dalam representasi heksadesimal (64 byte), dan
- `public_key` berupa kunci publik Ed25519 pengirim dalam representasi heksadesimal (32 byte).

Setelah diterima oleh Audit Receiver, ATS memverifikasi tanda tangan digital dengan menggunakan `public_key` yang disertakan, memastikan bahwa payload tidak mengalami perubahan sejak ditandatangani dan berasal dari entitas yang memiliki kunci privat yang bersesuaian. Apabila verifikasi berhasil, payload diurai menjadi objek AuditEvent yang diteruskan ke Audit Logger melalui antrian asinkron internal.

Setiap AuditEvent memuat atribut umum berikut yang wajib tersedia pada seluruh jenis *event*, mengacu pada kontrol AU-3 *Content of Audit Records* pada NIST SP

Tabel III.14: Atribut Umum Audit Event

Atribut	Deskripsi
source_component	Komponen sumber pembangkit <i>event</i> , misalnya nama modul atau layanan.
actor_id	Identitas aktor yang memicu aktivitas, seperti <i>IOTA address</i> atau identitas sesi.
target_object	Identitas objek atau sumber daya yang menjadi target aktivitas.
outcome	Hasil pelaksanaan aktivitas berupa Success, Failure, Denied, atau Unknown.
action_type	Jenis tindakan yang dilakukan.
event_type	Jenis <i>audit event</i> yang merujuk pada katalog <i>event</i> (EV1 hingga EV18) beserta atribut tambahan yang spesifik terhadap jenis tersebut.

Atribut `event_type` diimplementasikan sebagai *tagged enum* (`AuditEventDetails`) yang memuat atribut tambahan spesifik sesuai jenis *event*, sebagaimana telah diidentifikasi pada Tabel III.8.

III.2.3 Mekanisme Pembentukan dan Penyimpanan Audit Record

Setelah *event* diterima dari antrian internal, Audit Logger membentuk sebuah `AuditRecord` yang terdiri atas:

- `record_id` berupa pengenal unik berbasis UUID v7 yang mengandung informasi waktu sehingga bersifat monoton dan dapat diurutkan secara kronologis,
- `timestamp` berupa waktu pembentukan *record* dalam format UTC,
- `prev_record_hash` berupa nilai *hash* SHA-256 dari *record* sebelumnya, atau `null` apabila merupakan *record* pertama dalam rantai,
- `record_hash` berupa nilai *hash* SHA-256 dari kombinasi `record_id`, `timestamp`, `prev_record_hash`, dan isi *event*, serta
- seluruh atribut `AuditEvent` yang telah diverifikasi.

Mekanisme `record_hash` yang menyertakan `prev_record_hash` membentuk rantai *hash* (*hash chain*) antar *record* yang bersifat *append-only*: penambahan, modifikasi, atau penghapusan *record* pada posisi mana pun dalam rantai akan menyebabk-

an nilai `record_hash` seluruh *record* sesudahnya menjadi tidak konsisten, sehingga kerusakan dapat terdeteksi. Setiap `AuditRecord` yang terbentuk kemudian ditulis secara *append* ke berkas log lokal dalam format JSON-Lines (satu objek JSON per baris), menjamin bahwa berkas log tidak pernah menimpa entri yang sudah ada.

III.2.4 Mekanisme Rotasi, Pengarsipan, dan Publikasi Metadata

Log Rotation & Archival Worker berjalan secara periodik pada interval yang dapat dikonfigurasi. Pada setiap siklus rotasi, worker melakukan tiga tahap berurutan:

Tahap 1: Rotasi berkas log. Worker mengganti nama berkas log aktif dengan nama berkas arsip baru yang menyertakan *timestamp* rotasi. Operasi penggantian nama ini bersifat atomik pada sistem berkas sehingga tidak terjadi kehilangan *record* yang sedang ditulis.

Tahap 2: Pengarsipan ke IPFS. Worker menghitung nilai *hash* SHA-256 dari berkas arsip sebagai *fingerprint* integritas berkas, kemudian mengunggah berkas tersebut ke IPFS Cluster melalui endpoint `/add`. Proses unggah menghasilkan CID yang bersifat unik terhadap isi berkas; apabila isi berkas berubah, CID yang dihasilkan juga akan berbeda.

Tahap 3: Publikasi metadata ke IOTA. Worker menyusun objek `IotaLogMetadata` yang memuat informasi rotasi, termasuk CID hasil unggah IPFS, nilai *hash* berkas, nomor urut rotasi (`log_sequence_number`), waktu rotasi, dan referensi *transaction digest* rotasi sebelumnya (`prev_tx_digest`) untuk membentuk rantai antar entri metadata *on-chain*. Metadata tersebut kemudian diterbitkan ke jaringan IOTA melalui Gas Station Server yang telah ada, menghasilkan *transaction digest* baru yang bersifat *immutable*.

Kombinasi CID di IPFS dan *transaction digest* di IOTA memberikan dua lapis jaminan bagi setiap berkas arsip log. Isi berkas tidak dapat diubah tanpa mengubah CID-nya, dan CID beserta metadata yang telah diterbitkan ke IOTA tidak dapat diubah maupun dihapus setelah transaksi final. Dengan demikian, pihak yang berwenang dapat memverifikasi keutuhan suatu berkas arsip log secara independen dengan mengambil metadata dari IOTA, mengambil berkas dari IPFS menggunakan CID yang tercatat, menghitung ulang nilai *hash* SHA-256 berkas, dan memban-

dingkannya dengan nilai `file_hash` yang tersimpan pada metadata IOTA.

III.2.5 Pemenuhan Kebutuhan Sistem

Tabel III.15 menunjukkan pemetaan antara kebutuhan fungsional dan non-fungsional yang telah dirumuskan pada Tabel III.12 dan Tabel III.13 dengan mekanisme implementasi pada ATS yang dirancang.

Tabel III.15: Pemetaan Kebutuhan ke Mekanisme Rancangan ATS

ID	Kebutuhan	Mekanisme pada ATS
FR-01	Menerima seluruh audit event sesuai katalog.	Endpoint <code>POST /api/events</code> menerima seluruh jenis event EV1 hingga EV18 melalui format <code>SignedEvent</code> yang seragam.
FR-02	Membangkitkan audit record secara otomatis.	Audit Logger membentuk <code>AuditRecord</code> secara otomatis untuk setiap event yang diterima dari antrian internal.
FR-03	Struktur dan atribut audit record yang konsisten.	Seluruh <i>record</i> mengikuti skema <code>AuditRecord</code> yang seragam dengan atribut wajib dan <i>tagged enum</i> untuk atribut tambahan.
FR-04	Menyimpan audit record pada repositori terdedikasi.	Record ditulis ke berkas log lokal (<code>JSON-Lines</code>), lalu diarsipkan ke IPFS dengan referensi metadata di IOTA.
FR-05	Mengambil audit record untuk proses audit dan investigasi.	Metadata rotasi yang tersimpan di IOTA menyediakan CID untuk mengambil berkas arsip lengkap dari IPFS.

ID	Kebutuhan	Mekanisme pada ATS
FR-06	Mendeteksi kegagalan proses pembangkitan dan penyimpanan.	Kegagalan pada setiap tahap, yaitu verifikasi, penulisan, unggah IPFS, dan publikasi IOTA, dicatat ke <i>stderr</i> dan siklus rotasi dilanjutkan pada interval berikutnya.
FR-07	Pengelolaan retensi audit record.	Berkas arsip yang telah diunggah ke IPFS dihapus dari penyimpanan lokal sementara. Retensi jangka panjang bergantung pada kebijakan <i>pinning</i> pada IPFS Cluster.
NFR-01	Integritas setiap audit record.	Mekanisme <i>hash-chaining</i> , yaitu <i>record_hash</i> yang menyertakan <i>prev_record_hash</i> , menjamin deteksi modifikasi.
NFR-02	Keaslian (<i>authenticity</i>) audit record.	Verifikasi tanda tangan digital Ed25519 atas setiap <i>payload</i> event sebelum <i>record</i> dibentuk.
NFR-03	Timestamp yang akurat dan konsisten.	Setiap <i>record</i> menggunakan <i>Utc::now()</i> pada saat pembentukan <i>record</i> oleh Audit Logger.
NFR-04	Mekanisme <i>append-only</i> .	Penulisan ke berkas log menggunakan mode <i>append</i> ; mekanisme <i>hash-chaining</i> membuat modifikasi entri lama terdeteksi.
NFR-05	Retensi audit record.	Arsip log disimpan secara permanen di IPFS melalui mekanisme <i>pinning</i> dan referensinya tercatat secara <i>immutable</i> di IOTA.

ID	Kebutuhan	Mekanisme pada ATS
NFR-06	Dukungan <i>non-repudiation</i> .	Tanda tangan digital Ed25519 pada setiap <i>event</i> membuktikan bahwa <i>payload</i> dibuat oleh entitas yang memiliki kunci privat bersesuaian, dan <i>hash-chaining</i> membuktikan urutan kejadian tidak dapat diubah.

BAB IV

IMPLEMENTASI DAN PENGUJIAN

Bab ini menjelaskan proses implementasi dari rancangan solusi yang telah dipaparkan pada Bab III, khususnya penambahan layanan *Audit Trail Service* (ATS) dan integrasinya ke dalam arsitektur sistem DecMed yang telah ada.

IV.1 Lingkungan Implementasi

Implementasi dilakukan pada lingkungan yang sama dengan yang digunakan pada penelitian DecMed sebelumnya, sebagaimana dijelaskan pada Subbab I.4. Komponen ATS berjalan sebagai layanan *backend* Rust yang berdiri sendiri di lingkungan lokal bersama PRE Server. Daftar dependensi utama yang digunakan pada implementasi ATS ditunjukkan pada Tabel IV.1.

Tabel IV.1: Dependensi Utama Audit Trail Service

Dependensi	Versi	Fungsi
axum	0.8.4	<i>Web framework</i> HTTP untuk endpoint penerimaan <i>audit event</i> .
tokio	1.44.2	<i>Async runtime</i> Rust untuk pemrosesan konkuren.
serde / serde_json	1.x	Serialisasi dan deserialisasi data JSON.
ed25519-dalek	3.0.0	Verifikasi tanda tangan digital Ed25519 pada <i>signed event</i> .
sha2	0.10	Komputasi <i>hash</i> SHA-256 untuk mekanisme <i>hash-chaining</i> .
uuid	1.x	Pembangkitan <i>record_id</i> berbasis UUID v7 yang bersifat monoton.
chrono	0.4.39	Penanganan waktu dan <i>timestamp</i> dalam format UTC.
iota-sdk	1.2.0-alpha	Interaksi dengan jaringan IOTA untuk publikasi metadata rotasi log.
reqwest	0.12	Klien HTTP untuk unggah berkas ke IPFS dan interaksi dengan Gas Station.
tokio-util	0.7.11	<i>Streaming</i> berkas saat unggah ke IPFS menggunakan <i>BytesCodec</i> .

Pada PRE Server yang telah ada, dependensi *ed25519-dalek* versi 2.x ditambakk-

an untuk mendukung pembangkitan pasangan kunci penandatanganan event pada sisi pengirim.

IV.2 Implementasi

Implementasi sistem audit trail pada penelitian ini terdiri atas dua bagian utama. Pertama, implementasi komponen-komponen internal ATS sebagai layanan *backend* yang berdiri sendiri. Kedua, implementasi integrasi ATS ke dalam PRE Server yang telah ada pada arsitektur DecMed. Daftar komponen yang diimplementasikan beserta keterkaitan dengan rancangan ditunjukkan pada Tabel IV.2.

Tabel IV.2: Daftar Implementasi Sistem Audit Trail

ID	Komponen	Merealisasikan Rancangan
I-ATS-01	Skema tipe data AuditEvent dan AuditRecord.	Katalog <i>audit event</i> (EV1–EV18) dan skema atribut audit pada Tabel III.7–III.8.
I-ATS-02	Endpoint POST /api/events dan mekanisme verifikasi <i>signed event</i> .	Komponen Audit Receiver pada rancangan ATS.
I-ATS-03	Komponen AuditLogger dengan antrian asinkron internal.	Komponen Audit Logger dengan mekanisme <i>hash-chaining</i> .
I-ATS-04	Worker rotasi log, unggah IPFS, dan publikasi metadata ke IOTA.	Komponen Log Rotation & Archival Worker.
I-ATS-05	Modul ats pada PRE Server dan ATSCient.	Integrasi ATS ke dalam arsitektur DecMed.

IV.2.1 Implementasi Skema Tipe Data Audit

Implementasi skema tipe data audit merealisasikan katalog *audit event* dan skema atribut yang telah ditentukan pada Subbab ???. Seluruh tipe data didefinisikan pada berkas `audit-trail/src/types.rs`.

Tipe data AuditEvent mewakili satu peristiwa audit yang diterima dari komponen pengirim. Tipe ini memuat atribut umum yang wajib ada pada seluruh jenis *event* serta atribut tambahan yang spesifik terhadap jenis *event* melalui mekanisme *tagged enum* AuditEventDetails. Pendekatan ini memungkinkan satu struktur

data tunggal merepresentasikan seluruh 18 jenis *event* (EV1–EV18) yang telah diidentifikasi, dengan validasi tipe yang dijalankan oleh *compiler* Rust pada waktu kompilasi.

AuditEventDetails diimplementasikan sebagai *tagged enum* dengan anotasi `#[serde(tag = "event_type")]` sehingga nilai `event_type` pada JSON yang masuk menentukan varian enum yang akan diparsing. Setiap varian memuat atribut tambahan yang spesifik terhadap jenis *event* sesuai Tabel III.8. Sebagai contoh, varian EV6 untuk *event* permintaan layanan PRE memuat atribut `endpoint_called`, `request_id`, `caller_component`, dan `channel_encryption`.

Tipe data AuditRecord mewakili catatan audit yang telah terbentuk dan siap disimpan. Tipe ini memuat `record_id` berbasis UUID v7, `timestamp` dalam format UTC, `prev_record_hash` untuk membentuk rantai *hash*, `record_hash` sebagai sidik jari *record*, serta seluruh atribut AuditEvent melalui mekanisme `#[serde(flatten)]`. Penggunaan `flatten` memastikan seluruh atribut *event* diserialisasi pada tingkat yang sama dengan atribut *record* sehingga berkas JSON-Lines yang dihasilkan mudah dibaca.

Tipe data SignedEvent mewakili struktur yang diterima dari komponen pengirim melalui endpoint HTTP. Tipe ini memuat tiga bidang: *payload* sebagai representasi JSON dari *audit event*, *signature* sebagai tanda tangan digital Ed25519 atas *payload* dalam representasi heksadesimal, dan *public_key* sebagai kunci publik pengirim dalam representasi heksadesimal.

AuditOutcome diimplementasikan sebagai enum dengan empat varian: *Success*, *Failure*, *Denied*, dan *Unknown*, merealisasikan atribut *outcome* pada skema atribut umum Tabel III.7.

IV.2.2 Implementasi Audit Receiver

Komponen Audit Receiver diimplementasikan melalui handler `handle_event` pada berkas `audit-trail/src/handlers.rs` dan *routing* pada `audit-trail/src/main.rs`. Endpoint `POST /api/events` menerima *request* berisi objek SignedEvent dalam format JSON.

Proses pada handler `handle_event` terdiri atas dua fase. Fase pertama adalah verifikasi sumber *event* melalui fungsi `Utils::verify_and_extract_event`. Fungsi

ini melakukan tiga langkah: mendekode kunci publik dari representasi heksadesimal menjadi `VerifyingKey Ed25519`, mendekode tanda tangan dari representasi heksadesimal menjadi `Signature Ed25519`, kemudian memverifikasi tanda tangan terhadap byte-byte *payload*. Apabila verifikasi berhasil, *payload* string diurai menjadi objek `AuditEvent`. Apabila verifikasi gagal, handler mengembalikan respons *error* tanpa memproses *event* lebih lanjut.

Fase kedua adalah memasukkan *event* yang telah terverifikasi ke dalam antrian asinkron internal melalui `state.audit_tx.send(audit_event).await`. Antrian ini diimplementasikan menggunakan `tokio::sync::mpsc` dengan kapasitas 10.000 *event*, memastikan ATS dapat menerima *event* dalam jumlah besar tanpa memblokir pengirim.

Struktur `Handlers` hanya menyimpan `audit_tx` bertipe `Sender<AuditEvent>` sebagai satu-satunya *state*. Desain ini dengan sengaja memisahkan proses penerimaan *event* dari proses pembentukan dan penyimpanan *audit record*, sehingga keduanya dapat berjalan secara konkuren tanpa saling memblokir.

IV.2.3 Implementasi Audit Logger

Komponen Audit Logger diimplementasikan sebagai struct `AuditLogger` pada berkas `audit-trail/src/audit.rs`. Komponen ini berjalan sebagai *task* Tokio terpisah yang di-*spawn* pada saat aplikasi dimulai, menerima *event* dari antrian asinkron, membentuk *audit record*, dan menulis ke berkas log.

`AuditLogger` menyimpan dua bidang: `rx` bertipe `Receiver<AuditEvent>` sebagai sisi penerima antrian asinkron, dan `prev_record_hash` bertipe `Option<String>` sebagai hash dari *record* sebelumnya untuk keperluan *hash-chaining*. Bidang `prev_record_hash` diinisialisasi dengan `None` pada *record* pertama dalam satu siklus kerja ATS.

Metode `run` menjalankan *loop* utama yang terus berjalan selama pengirim masih aktif, menggunakan pola `while let Some(event) = self.rx.recv().await`. Untuk setiap *event* yang diterima, *loop* melakukan tiga langkah berurutan. Pertama, memanggil fungsi `create_audit_record` untuk membentuk `AuditRecord` baru. Kedua, memanggil fungsi `write_audit_record` untuk menulis *record* ke berkas log. Ketiga, memperbarui `self.prev_record_hash` dengan *hash* dari *record* yang baru saja ditulis.

Fungsi `create_audit_record` merealisasikan mekanisme *hash-chaining* dengan langkah-langkah berikut: membangkitkan `record_id` baru menggunakan `Uuid::now_v7()`, mengambil waktu saat ini menggunakan `Utc::now()`, kemudian mendelegasikan komputasi *hash* ke `Utils::calculate_record_hash`. Fungsi `calculate_record_hash` menggabungkan `record_id`, `timestamp`, `prev_record_hash`, dan isi *event* menjadi satu objek JSON, kemudian menghitung SHA-256 dari representasi JSON tersebut dan mengembalikannya sebagai string heksadesimal.

Fungsi `write_audit_record` membuka berkas log dengan mode *append* menggunakan `OpenOptions::new().create(true).append(true)`, kemudian menulis representasi JSON dari *record* diikuti karakter baris baru (`\n`). Penggunaan mode *append* menjamin bahwa entri yang sudah ada tidak pernah tertimpa, merealisasikan karakteristik *append-only* yang disyaratkan oleh NFR-04. Konstanta `LOG_FILE_PATH` yang merujuk ke `./logs/audit_logs.log` didefinisikan pada `audit-trail/src/` dan digunakan sebagai satu-satunya referensi jalur berkas log di seluruh kode ATS.

IV.2.4 Implementasi Log Rotation dan Archival Worker

Komponen Log Rotation & Archival Worker diimplementasikan sebagai fungsi `Utils::spawn_log_rotation_worker` pada berkas `audit-trail/src/Utils.rs`. Fungsi ini menerima satu argumen `package_id` yang merupakan ID paket *smart contract* ATS pada jaringan IOTA, kemudian men-*spawn* sebuah *task* Tokio yang berjalan selamanya dengan interval yang dapat dikonfigurasi melalui konstanta `LOG_ROTATION_INTERVAL`. Setiap siklus rotasi, worker melakukan tiga tahap berurutan.

Tahap 1 – Rotasi berkas log. Worker memeriksa apakah berkas log aktif ada dan tidak kosong. Apabila berkas kosong, siklus dilewati. Apabila berkas berisi data, worker membentuk nama berkas arsip baru dengan format `logs/uploading_{timestamp}.log` menggunakan *Unix timestamp* saat ini, kemudian memindahkan berkas log aktif ke nama arsip baru menggunakan `fs::rename`. Operasi `rename` bersifat atomik pada sistem berkas Linux sehingga tidak terjadi kehilangan *record* yang sedang ditulis oleh Audit Logger.

Tahap 2 – Unggah ke IPFS. Worker menghitung nilai *hash* SHA-256 dari berkas arsip menggunakan `IotaLogClient::hash_file`, kemudian mengunggah berkas ke IPFS Cluster melalui fungsi `Utils::add_file_to_ipfs`. Fungsi ini menggu-

nakan FramedRead dengan BytesCodec untuk melakukan *streaming* berkas tanpa memuat seluruh konten ke memori sekaligus, yang penting untuk berkas log berukuran besar. Respons dari IPFS menghasilkan CID yang menjadi penanda konten berkas.

Tahap 3 – Publikasi metadata ke IOTA. Worker membentuk objek `IotaLogMetadata` yang memuat informasi rotasi, termasuk CID hasil unggah IPFS, nilai *hash* berkas (`file_hash`), nomor urut rotasi (`log_sequence_number`), waktu rotasi, dan referensi *transaction digest* rotasi sebelumnya (`prev_tx_digest`). Bidang `prev_tx_digest` membentuk rantai antar entri metadata *on-chain*, analog dengan mekanisme *hash-chaining* pada *audit record*. Metadata tersebut kemudian diterbitkan ke jaringan IOTA melalui `IotaLogClient::publish_metadata`.

Implementasi `IotaLogClient` pada berkas `audit-trail/src/iota_client.rs` menangani seluruh interaksi dengan jaringan IOTA. Fungsi `publish_metadata` melakukan serangkaian langkah: menserialisasi metadata ke JSON string, membangun transaksi IOTA menggunakan `ProgrammableTransactionBuilder`, mere Reservasi *gas* dari Gas Station Server, menandatangani transaksi dengan kunci privat ATS, kemudian mengeksekusi transaksi melalui Gas Station Server. Apabila transaksi berhasil, fungsi mengembalikan `PublishResult` yang memuat `object_id` objek `LogRecord` yang dibuat *on-chain* serta `tx_digest` transaksi.

Smart contract Move yang digunakan ATS didefinisikan pada `move/ats/sources/audit_trail.mo`. Modul `ats::audit_log` menyediakan fungsi `create_log` yang menerima string JSON metadata kemudian membuat objek `LogRecord` yang di-*freeze* sehingga bersifat *immutable* setelah dibuat. Pembekuan objek (`transfer::freeze_object`) adalah mekanisme utama yang menjamin sifat *immutability* metadata rotasi log pada jaringan IOTA, merealisasikan NFR-01 dan NFR-05.

Setelah publikasi ke IOTA berhasil, worker memperbarui `prev_tx_digest` untuk siklus berikutnya dan menghapus berkas arsip dari penyimpanan lokal menggunakan `fs::remove_file`. Retensi jangka panjang bergantung pada mekanisme *pinning* IPFS Cluster yang telah digunakan DecMed.

IV.2.5 Implementasi Integrasi ATS ke DecMed

Integrasi ATS ke dalam arsitektur DecMed dilakukan secara seragam pada seluruh komponen sumber *event* (*event source*) yang telah diidentifikasi pada Subbab II.3.4, yaitu PRE Server, Patient Client, Hospital Client, dan Ministry Client. Setiap *event source* diberikan penambahan modul *ats* pada direktori sumber masing-masing komponen (mis. `proxy-reencryption/src/ats/` untuk PRE Server), dengan pola struktur yang konsisten di setiap komponen: berkas `ats.rs` yang memuat implementasi utama dan `mod.rs` yang mengeksport tipe-tipe publik.

Tipe data bersama. Pada setiap *event source*, berkas `ats.rs` mendefinisikan ulang tipe-tipe data `AuditEvent`, `AuditOutcome`, dan `AuditEventDetails` yang strukturnya setara dengan yang ada di ATS. Pendekatan ini dipilih agar setiap komponen sumber *event* tidak bergantung pada *crate* ATS secara langsung, mempertahankan independensi antarkomponen sesuai arsitektur DecMed yang terdiri atas layanan-layanan yang terpisah. Varian `AuditEventDetails` yang diimplementasikan pada masing-masing *event source* dibatasi hanya pada jenis *event* yang relevan dengan aktivitas komponen tersebut. Tipe `SignedAuditEvent` mendefinisikan struktur JSON yang dikirimkan oleh seluruh *event source* ke endpoint `POST /api/events` pada ATS, sehingga format pengiriman tetap seragam meskipun isi `AuditEventDetails` berbeda-beda antarkomponen.

Implementasi ATSCClient. Setiap modul *ats* pada masing-masing *event source* menyediakan struct `ATSCClient` yang bertanggung jawab mengelola pasangan kunci Ed25519 untuk penandatanganan *event* serta mengirimkan *signed event* ke ATS. Kunci Ed25519 dibangkitkan secara acak pada saat inisialisasi menggunakan `SigningKey::generate(&mut OsRng)`, sehingga setiap instans komponen memiliki kunci penandatanganan yang unik terlepas dari jenis komponennya. Kunci publik yang bersesuaian disertakan pada setiap *signed event* agar ATS dapat memverifikasi tanda tangan tanpa perlu menyimpan daftar kunci yang diizinkan terlebih dahulu, sehingga penambahan *event source* baru di kemudian hari tidak memerlukan konfigurasi tambahan pada sisi ATS.

`ATSCClient` menyediakan dua metode pengiriman yang sama pada seluruh *event source*. Metode `send_event` bersifat *async* dan menunggu respons dari ATS, berguna ketika kode pemanggil perlu menangani *error* pengiriman. Metode `send_event_nonblocking`

men-*spawn task* Tokio terpisah yang berjalan secara *fire-and-forget*; apabila pengiriman gagal, *error* hanya dicatat ke *stderr* tanpa memblokir alur utama. Metode kedua inilah yang digunakan pada seluruh instrumentasi di setiap *event source* agar operasi audit tidak menambah latensi pada alur layanan utama masing-masing komponen.

Implementasi *ATSTWorker* sebagai *helper* internal menangani proses penandatanganan dan pengiriman pada *spawned task*, dan direplikasi dengan pola yang sama pada setiap modul *ats*. Worker ini menyalin byte-byte kunci penandatanganan dari *ATSTClient* agar dapat merekonstruksi *SigningKey* di dalam task yang bersifat *'static*. Proses penandatanganan menggunakan `self.signing_key.sign(&payload.bytes)` dari *trait* *Signer* pada *crate* *ed25519-dalek*, menghasilkan tanda tangan 64-byte yang kemudian dikodekan sebagai heksadesimal.

Integrasi ke *state* komponen. Pada setiap *event source*, sebuah bidang *ats_client*: *ATSTClient* ditambahkan pada struct *state* utama komponen tersebut (misalnya *AppState* pada PRE Server di *proxy-reencryption/src/types.rs*, atau struct *state/context* yang setara pada *Patient Client*, *Hospital Client*, dan *Ministry Client*). Inisialisasi dilakukan pada titik masuk (*entry point*) masing-masing komponen dengan membaca *ATS_ENDPOINT* dari *environment variable*, kemudian memanggil `ATSTClient::with_endpoint(&a`. Nilai *default* endpoint adalah `http://localhost:3000/api/events` apabila variabel tidak dikonfigurasi, sehingga seluruh *event source* pada lingkungan implementasi mengarah ke instans *ATS* yang sama.

Distribusi event per *event source*. Berbeda dengan PRE Server yang berjalan sebagai layanan *backend*, *Patient Client*, *Hospital Client*, dan *Ministry Client* merupakan aplikasi *client* berbasis Tauri. Pada ketiga aplikasi tersebut, modul *ats* diimplementasikan pada sisi *backend* Rust dari aplikasi Tauri (bukan pada lapisan antarmuka *SvelteKit/TypeScript*), sehingga penandatanganan dan pengiriman *event* tetap dilakukan dengan kunci yang tidak terekspos ke lapisan antarmuka. Jumlah dan jenis *event* yang diproduksi oleh setiap komponen berbeda-beda sesuai dengan aktivitas yang menjadi tanggung jawabnya, sebagaimana dirangkum pada Tabel IV.3. Kode EV pada kolom ketiga tabel tersebut merujuk pada daftar *audit event* hasil identifikasi pada Tabel ??.

Perlu dicatat bahwa permintaan dan keputusan sponsorship transaksi (EV9, EV10)

Tabel IV.3: Pemetaan Event Source terhadap Audit Event yang Diproduksi

Event Source	Jumlah Event	Kode Audit Event
PRE Server	11	EV6, EV7, EV8, EV9, EV10, EV13, EV14, EV15, EV16, EV17, EV18
Patient Client	9	EV1, EV2, EV3, EV8, EV9, EV10, EV11, EV12, EV17
Hospital Client	7	EV1, EV4, EV8, EV9, EV10, EV11, EV12
Ministry Client	4	EV5, EV8, EV9, EV10

serta pengiriman transaksi ke jaringan IOTA (EV8) tercatat pada sisi pemohon (*caller*) — yaitu PRE Server, Patient Client, Hospital Client, atau Ministry Client, tergantung komponen mana yang menginisiasi transaksi tersebut — bukan pada Gas Station Server itu sendiri. Gas Station Server pada rancangan ini diperlakukan sebagai infrastruktur pendukung yang tidak diinstrumentasi sebagai *event source* tersendiri, sehingga tidak memiliki modul *ats*; keputusan sponsorship yang dihasilkannya cukup direkam oleh komponen pemanggil sebagai bagian dari EV9/EV10.

Seluruh instrumentasi pada Tabel IV.3 menggunakan metode `send_event_nonblocking` agar tidak menambah latensi pada alur utama masing-masing *event source*. Variabel *environment* `ATS_ENDPOINT` dikonfigurasi secara konsisten pada berkas `.env` setiap komponen (mis. `proxy-reencryption/.env` untuk PRE Server, dan berkas konfigurasi setara pada ketiga aplikasi *client*) ke `http://localhost:3000/api/events`, sehingga seluruh *event source* mengirimkan *event* ke instans ATS yang sama di lingkungan lokal. Pendekatan modul *ats* yang seragam ini memungkinkan penambahan *event source* baru di masa mendatang dilakukan dengan mereplikasi pola modul yang sama tanpa mengubah desain ATS itu sendiri.

IV.2.6 Implementasi Verifikasi Integritas Audit Record

Fungsi `Utils::verify_record` pada berkas `audit-trail/src/Utils.rs` merealisasikan mekanisme verifikasi integritas satu *audit record*. Fungsi ini menghitung ulang `record_hash` dari `record_id`, `timestamp`, `prev_record_hash`, dan isi *event* menggunakan `calculate_record_hash`, kemudian membandingkannya dengan `record_hash` yang tersimpan. Kesamaan nilai keduanya membuktikan bahwa *record* tidak mengalami modifikasi sejak dibuat.

Verifikasi rantai secara penuh dilakukan dengan memeriksa setiap *record* secara

ra berurutan: nilai `prev_record_hash` setiap *record* harus sama persis dengan `record_hash` dari *record* sebelumnya. Apabila terdapat *record* yang dihapus, ditambahkan, atau dimodifikasi di tengah rantai, nilai `record_hash` seluruh *record* sesudahnya akan menjadi tidak konsisten dan verifikasi akan gagal.

IV.3 Pengujian

[Bagian pengujian akan ditambahkan setelah implementasi selesai diverifikasi.]

IV.4 Pengujian

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

BAB V

PENUTUP

V.1 Kesimpulan

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

V.2 Saran

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

DAFTAR PUSTAKA

- Al Amin, M., Shah, R., Tummala, H., & Ray, I. (2025). Did You Break the Glass Properly? A Policy Compliance Framework for Protected Health Information (PHI) Emergency Access. *Proceedings of the 22nd International Conference on Security and Cryptography (SECRYPT 2025)*, 195–208.
- Benet, J. (2014). IPFS - Content Addressed, Versioned, P2P File System. <https://arxiv.org/abs/1407.3561>
- Hernández-Ramos, J. L., Jara, A. J., Marin, L., & Skarmeta, A. F. (2013). Distributed Capability-Based Access Control for the Internet of Things. *Journal of Internet Services and Information Security (JISIS)*, 3(3/4), 1–16.
- International Organization for Standardization. (2021). ISO 27789:2021 – Health Informatics – Audit Trails for Electronic Health Records (Second edition). <https://www.iso.org/standard/76028.html>
- International Organization for Standardization. (2022). ISO/IEC 27002:2022 – Information Security, Cybersecurity and Privacy Protection – Information Security Controls [Control 8.15: Logging]. <https://www.iso.org/standard/75652.html>
- Islam, M. S., & Rahman, M. S. (2025). LogStamping: A Blockchain-Based Log Auditing Approach for Large-Scale Systems. <https://arxiv.org/abs/2505.17236>
- Peraturan Menteri Kesehatan Republik Indonesia Nomor 24 Tahun 2022 tentang Rekam Medis (2022). <https://peraturan.bpk.go.id/Details/245544/permenkes-no-24-tahun-2022>
- Kent, K., & Souppaya, M. (2006). *Guide to Computer Security Log Management* (techreport **number** NIST Special Publication 800-92). National Institute of Standards dan Technology. <https://doi.org/10.6028/NIST.SP.800-92>
- Laksono, A. C., & Prayudi, Y. (2021). Threat Modeling Menggunakan Pendekatan STRIDE dan DREAD untuk Mengetahui Risiko dan Mitigasi Keamanan pada Sistem Informasi Akademik. *JUSTINDO (Jurnal Sistem dan Teknologi Informasi Indonesia)*, 6(1), 9–20. <https://doi.org/10.32528/justindo.v6i1.3944>

- National Institute of Standards and Technology. (2020). *Security and Privacy Controls for Information Systems and Organizations* (techreport **number** NIST Special Publication 800-53, Revision 5). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.SP.800-53r5>
- Undang-Undang Republik Indonesia Nomor 19 Tahun 2016 tentang Perubahan atas Undang-Undang Nomor 11 Tahun 2008 tentang Informasi dan Transaksi Elektronik (2016). <https://peraturan.bpk.go.id/Details/37582/uu-no-19-tahun-2016>
- Pramesti, D. P. A., Ayuningtyas, D., & Verdi, R. (2024). Keamanan dan Kerahasiaan Data Medis Pasien dalam Implementasi Rekam Medis Elektronik: Tinjauan Sistematis. *PREPOTIF: Jurnal Kesehatan Masyarakat*, 8(3), 7691–7702. <https://doi.org/10.31004/prepotif.v8i3.38445>
- Purnama, H., Sudewa, I. P. B. H., Nizami, T., Rosyada, B. S., Mahesa, P. R., & Ahmadi, N. (2026). Access Control Development Within the Framework of an IOTA-Based Electronic Medical Record Management System. *Sensors*, 26(5), 1422.
- Theis, M., Trzeciak, R. F., Costa, D. L., Moore, A. P., Miller, S., Cassidy, T., & Claycomb, W. R. (2022). *Common Sense Guide to Mitigating Insider Threats* (techreport) (7th edition). CERT Division, Software Engineering Institute, Carnegie Mellon University.

Lampiran A. Contoh gambar pengujian



Gambar A.1: Contoh gambar